昆仑芯P800、英伟达H100 - 单节点 MiniMax-M2.5模型整体测试比对报告

*测试日期：2026-04-08 ~ 2026-05-08
*测试人员：九章云极

1. 测试背景

公司需要在多个候选开源大模型中选型，部署基于vLLM的推理服务。并需要在满足各项模型服务指标的情况下，选定芯片集采厂商。

2. 测试目标

本测试主要评估不同芯片在单机环境下运行大模型推理的能力，为后续集群采购和生产部署提供决策依据。

1. **硬件摸底**：确认各芯片型号实际规格（算力、显存、带宽）与标称值的一致性
2. **功能验证**：各模型在各芯片环境上的推理正确性和算子兼容性
3. **性能基准**：吞吐量、延迟、显存效率等关键指标
4. **单机极限**：8 卡 Tensor Parallel 的性能上限和资源利用率
5. **稳定性验证**：长时间运行下的可靠性
6. **K8S 容器化验证**：单节点 K8S 环境下GPU/DCU/XPU 调度、资源管理和服务编排能力

3. 测试环境

3.1 硬件规格

组件 \ 规格	英伟达	昆仑芯	状态
节点数量	1 台	1 台	确认
芯片型号	H100	P800 OAM	确认
芯片数量	8 张	8 张	确认
单卡算力 FP16/BF16	1979 TFLOPS （官方理论值）	待确认	⚠️ 待确认
单卡算力 FP32	67 TFLOPS （官方理论值）	待确认	⚠️ 待确认
单卡算力 FP64	34 TFLOPS （官方理论值）	待确认	⚠️ 待确认
单卡显存	80GB	96GB	确认
显存类型	HBM3	待确认	⚠️ 待确认
显存带宽	3.35 TB/s	待确认	⚠️ 待确认
单卡功耗	700 W	400 W	确认
卡间互联	NVLink 4.0	XPULink (XL) + PCIe Gen4 x16（跨 NUMA 显示 SYS）	确认
CPU	Intel(R) Xeon(R) Platinum 8468 (192核)	Intel Xeon Platinum 8563C (208核)	确认
系统内存	2.0 TiB	2.0 TiB	确认
本地存储	894GB 系统盘 + 7TB*4 缓存盘 + 7TB 容器盘 + 25TB 扩展盘	446.6G + NVMe 4 x 3.5T (Intel SSDPF2KX038T1)	确认

3.2 软件栈

组件\版本	英伟达	昆仑芯	说明
操作系统	Ubuntu 22.04.5 LTS	Ubuntu 22.04.5 LTS	芯片所在物理机系统
显卡驱动	570.133.20/580.126.09	Kunlun / XPU Driver 5.0.21.26	驱动信息
Toolkit	release 12.9	XPU Container Runtime Hook version 1.0.5	CUDA Toolkit版本
Docker	-	28.4.0	容器运行时
containerd	2.2.0	1.7.28	K8S 容器运行时（CRI）
Kubernetes	1.34.2	v1.28.2	单节点 All-in-One 部署
Device Plugin	0.14.5	xpu-device-plugin v5.0.0-alpha.2	K8S GPU 资源管理
多卡通信库	NCCL	无单独查询版本	多卡通信库

3.3 模型配置信息

参数名称	NVIDIA_H100	Kunlun_P800
model_name	MiniMax-M2.5	MiniMax-M2.5-W8A8-INT8-Dynamic
quantization_config	FP8	int-8
model_size	215G	215G
max_position_embeddings	196608	196608
temperature	1.0	1.0
top_k	40	40
top_p	0.95	0.95
transformers_version	4.46.1	4.46.1
vllm_version	0.20.0	0.11.0
python_version	3.12.3	3.10.10

3.4 推理框架主要启动参数

参数名称	NVIDIA_H100	Kunlun_P800
Model Name	MiniMax-M2.5	MiniMax-M2.5-W8A8-INT8-Dynamic
Max Model Len	196608	196608
Max Num Seqs	64	64
Max Num Batched Tokens	8192	8192
Block Size	default	128
Gpu Memory Utilization	0.85	0.95
Compilation Config	N/A	{"splitting_ops":["vllm.unified_attention", "vllm.unified_attention_with_output", "vllm.unified_attention_with_output_kunlun", "vllm.mamba_mixer2", "vllm.mamba_mixer", "vllm.short_conv", "vllm.linear_attention", "vllm.plamo2_mamba_mixer", "vllm.gdn_attention", "vllm.sparse_attn_indexer", "vllm.sparse_attn_indexer_vllm_kunlun"]}
Dtype	default	auto
Dp	1	1
Tp	8	8
Pp	1	1
Reasoning Parser	minimax_m2	minimax_m2
Tool Call Parser	minimax_m2	minimax_m2
Enable Auto Tool Choice	True	True
Enable Export Parallel	True	False

4. 测试场景及概况

4.1 测试场景列表

序号	测试场景
场景一	vllm benchmark基准测试
场景二	单、多并发超长上下文请求
场景三	多并发长上下文极限验证
场景四	多I/O测试
场景五	模型精度测试
场景六	多轮对话测试
场景七	模型推理功能测试

4.2 模型部署问题汇总

对于昆仑芯的vLLM模型部署，在TP=8的情况下，若同时启动专家并行模式，这种情况下是双重高强度通信叠加，在 BKCL / XCCL 上，非常容易出现以下问题：

- buffer size 计算错误
- rank 不一致
- 通信 hang / 越界 （实际测试确实遇到了该问题）

4.3 模型推理测试问题汇总

对于思考模式的启动参数，昆仑芯环境部署按照--reasoning-parser minimax_m2启动后，响应内容依然是直接写入到content里，而不是写入到think标签里。
参数变更为 --reasoning-parser_appened_think minimax_m2后，think标签的内容会被合并到content里,由此带来另一个问题是tool_call工具调用功能失败。

以下是每个测试场景的详细结果报告

> Kunlun_P800平台，Minimax-M2.5模型benchmark测试脚本见本报告 **《附录一》**

测试场景一：vllm benchmark基准测试

测试目标：在相同请求数、基础长度上下文参数下，使用vllm bench serve工具对并发数逐级增加场景的性能基准验证.

主要采集指标：

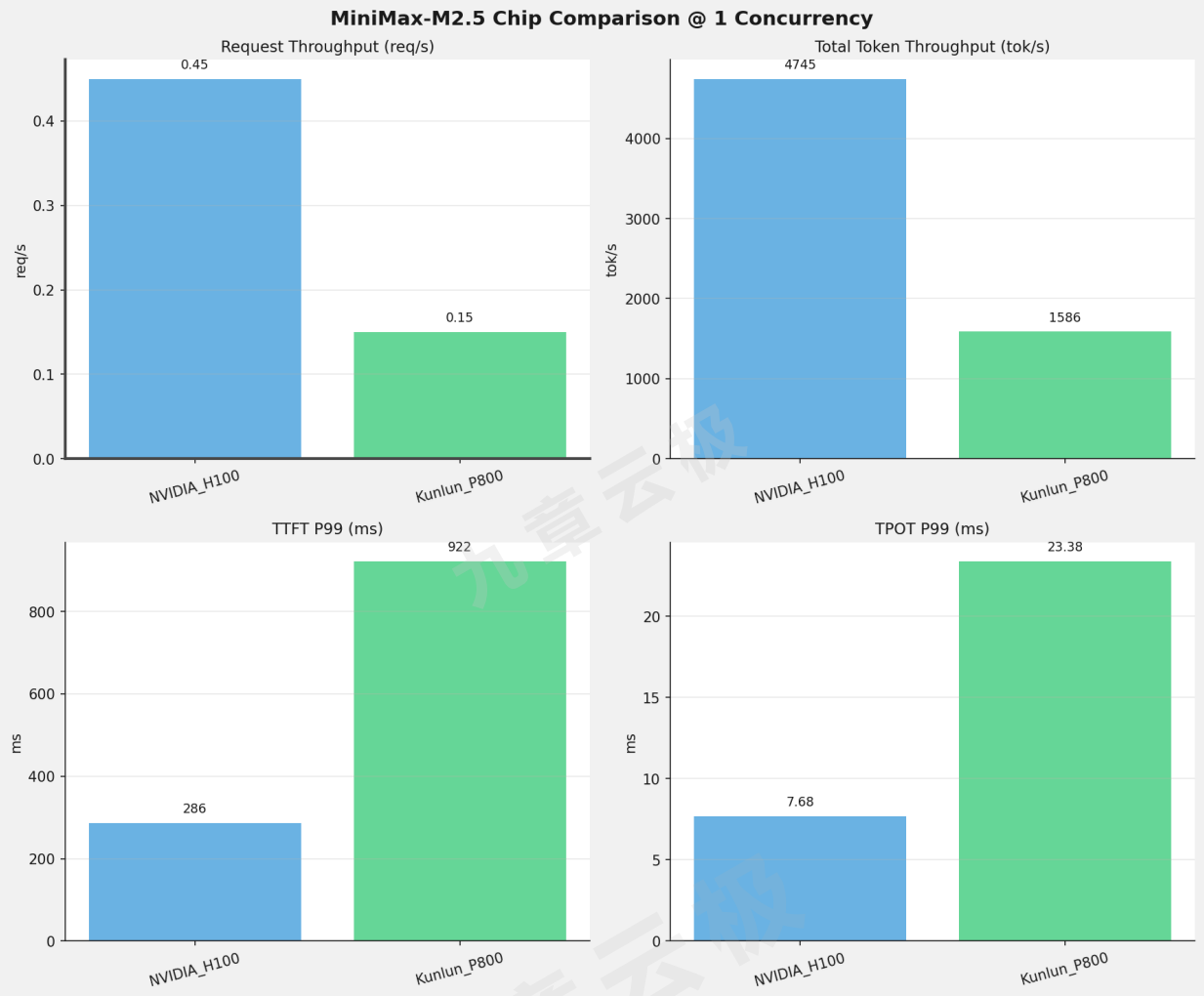
指标	单位	含义
TTFT	ms	Time To First Token，首 token 延迟
TPOT	ms/token	Time Per Output Token，每 token 生成时间
Throughput	tokens/s	系统总吞吐
QPS	requests/s	请求吞吐
P95/P99 Latency	ms	延迟分位数

📊 测试概览

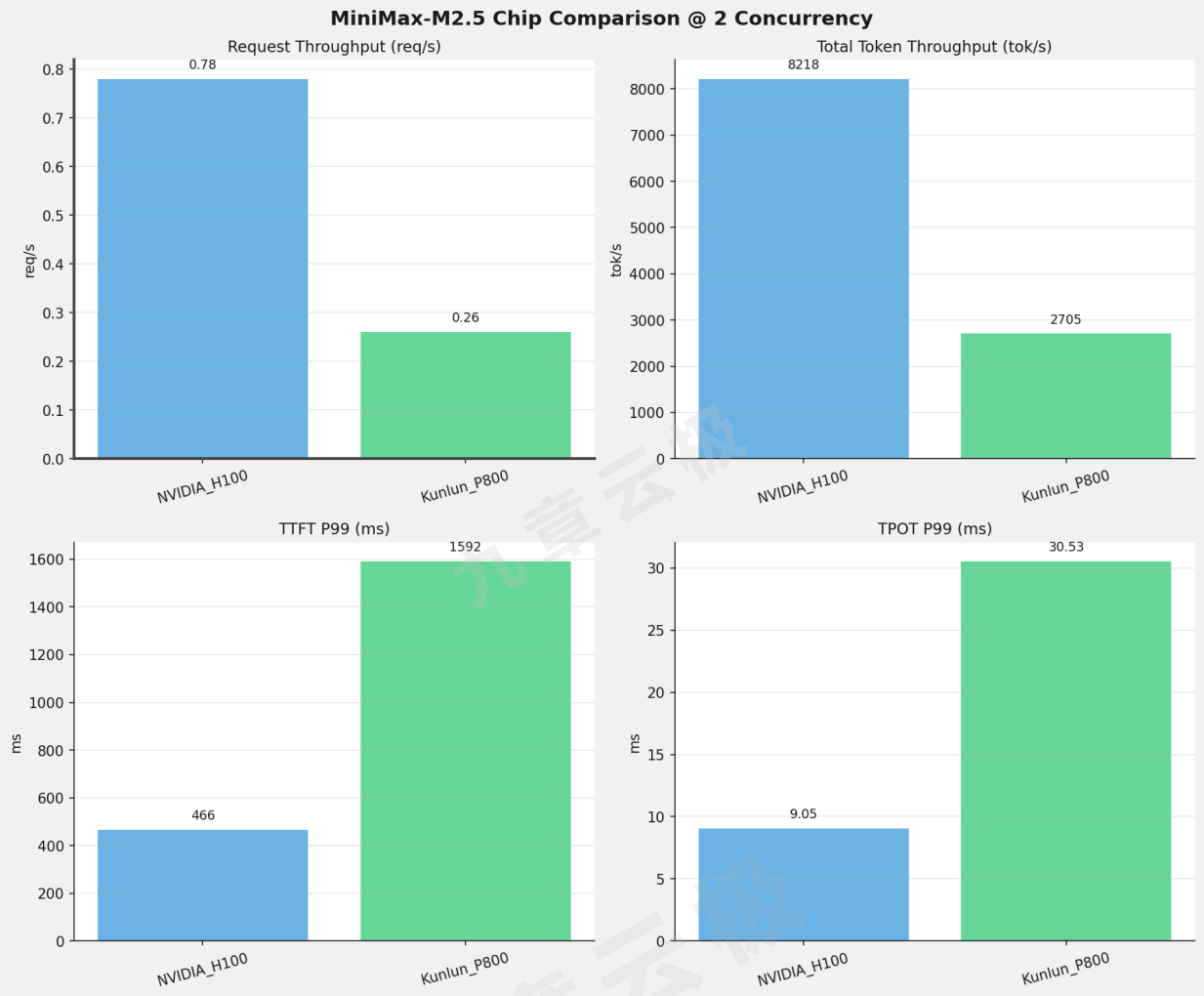
项目	配置	备注
数据集	random	
并发数	1, 2, 4, 8, 10, 16, 32, 64, 80, 128	
总请求数	320	
请求输入上下文长度	10240（10k）	
请求输出上下文长度	256（0.25k）	
被测芯片	NVIDIA_H100, Kunlun_P800	
被测模型	NVIDIA_H100 (MiniMax-M2.5); Kunlun_P800 (MiniMax-M2.5-W8A8-INT8-Dynamic)	

📊 芯片性能对比柱状图

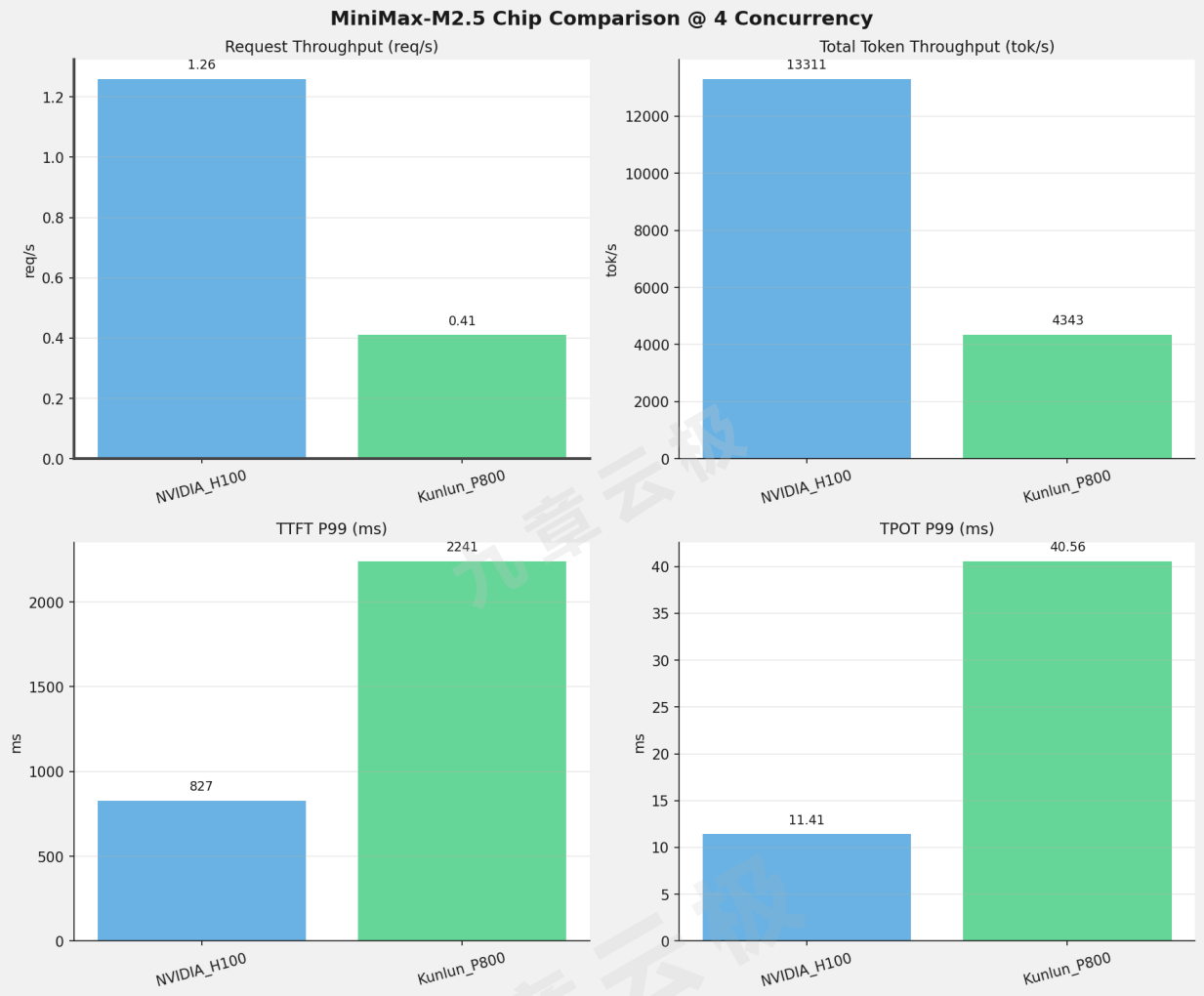
1并发


MiniMax-M2.5 Chip Comparison @ 1 Concurrency
Request Throughput (req/s)
req/s
0.45
0.15
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
4745
1586
TTFT P99 (ms)
ms
286
922
TPOT P99 (ms)
7.68
23.38

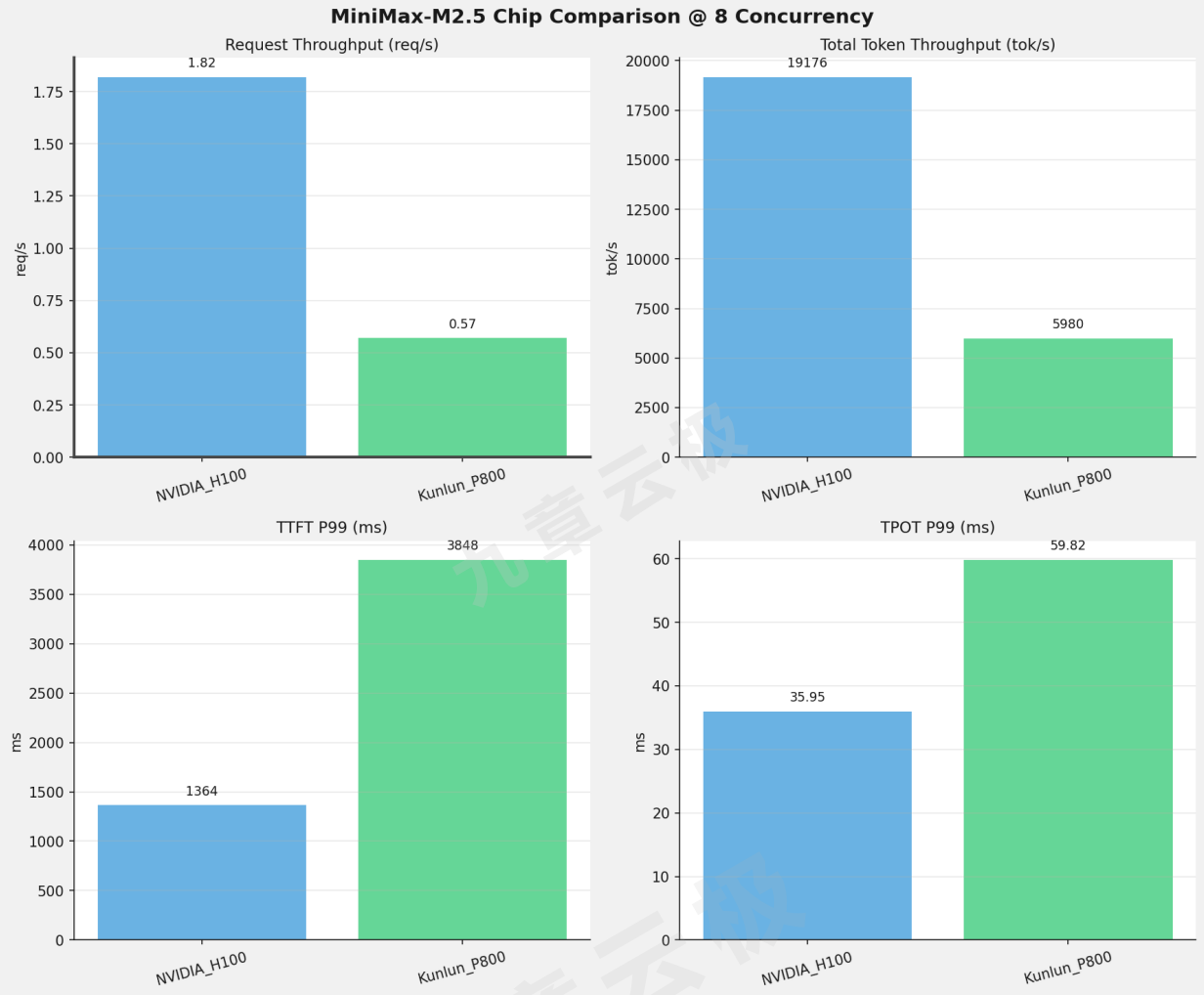
2并发


MiniMax-M2.5 Chip Comparison @ 2 Concurrency
Request Throughput (req/s)
req/s
0.78
0.26
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
8218
2705
NVIDIA_H100
Kunlun_P800
TTFT P99 (ms)
ms
466
1592
NVIDIA_H100
Kunlun_P800
TPOT P99 (ms)
ms
9.05
30.53
NVIDIA_H100
Kunlun_P800

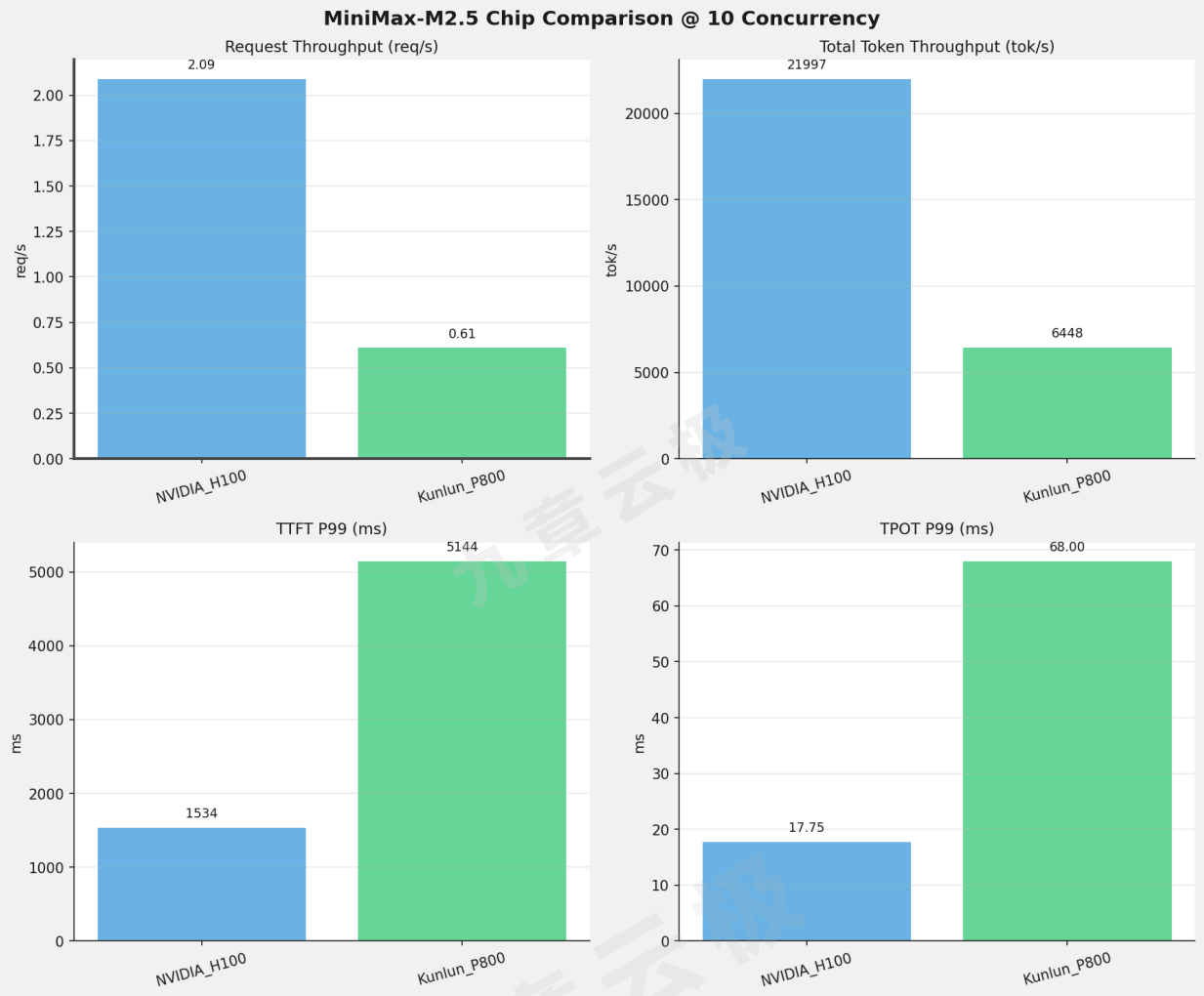
4并发


MiniMax-M2.5 Chip Comparison @ 4 Concurrency
Request Throughput (req/s)
req/s
1.26
0.41
Total Token Throughput (tok/s)
tok/s
13311
4343
TTFT P99 (ms)
ms
827
2241
TPOT P99 (ms)
ms
11.41
40.56
NVIDIA_H100
Kunlun_P800

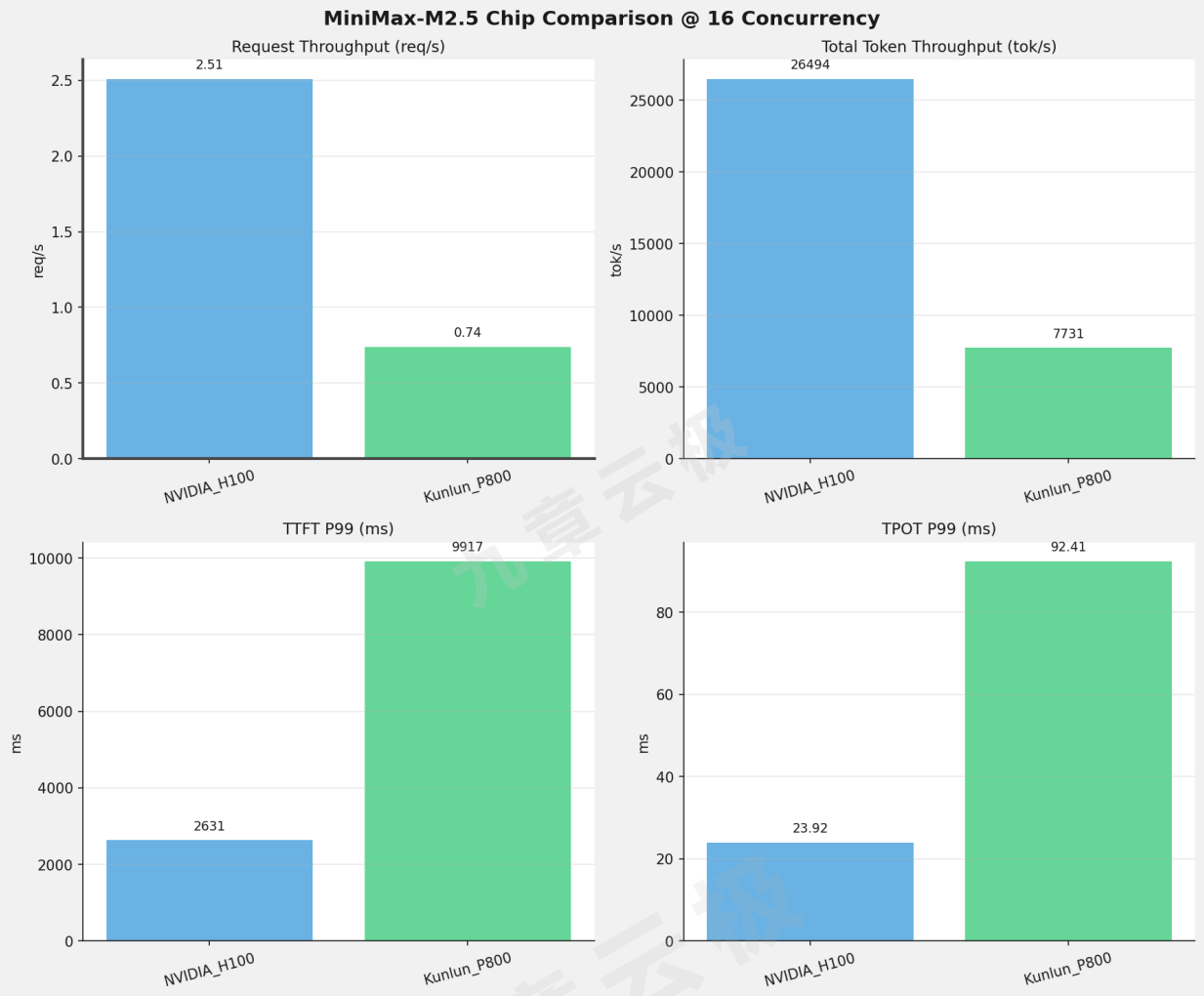
8并发



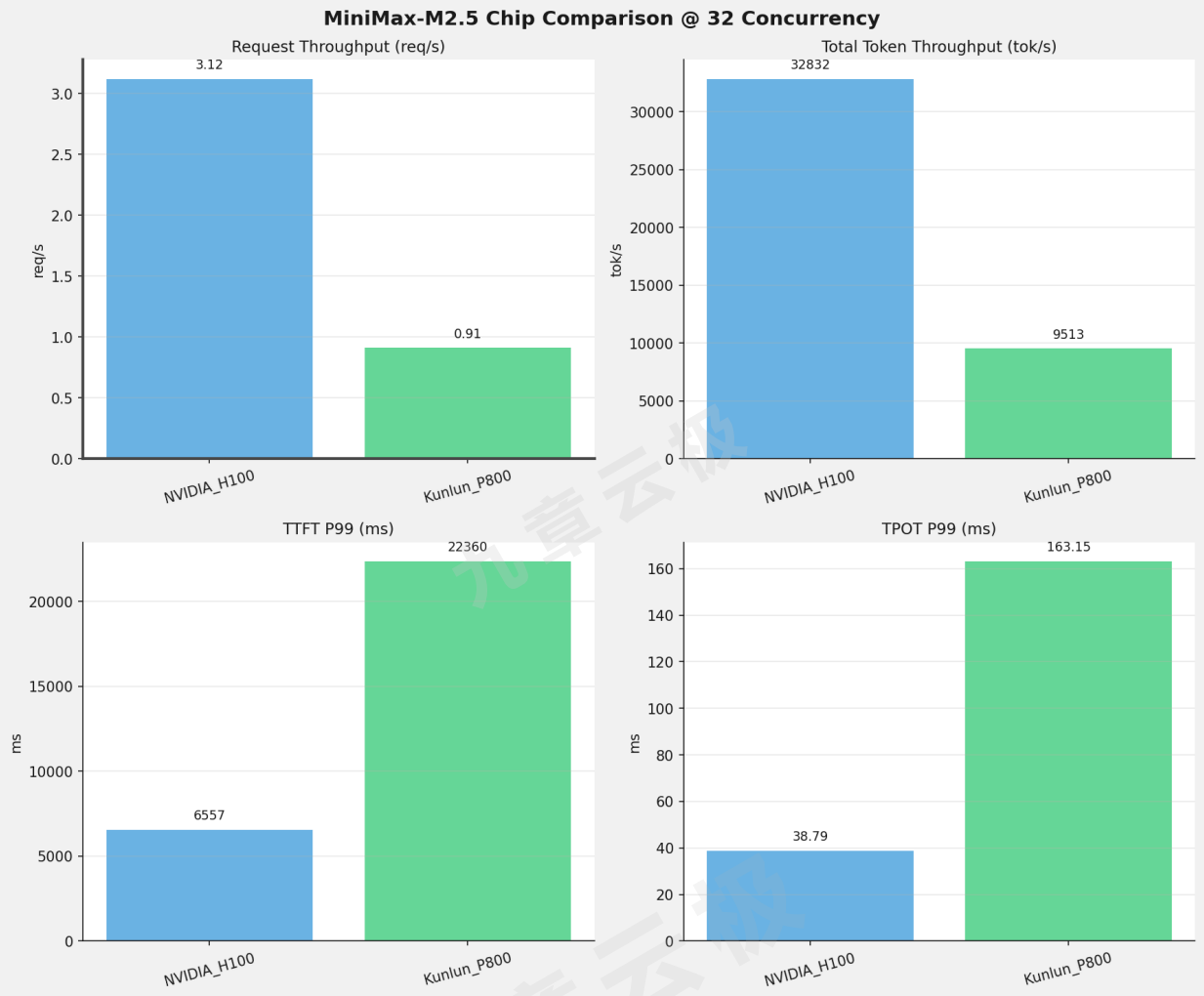
10并发


MiniMax-M2.5 Chip Comparison @ 10 Concurrency
Request Throughput (req/s)
req/s
2.09
0.61
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
21997
6448
NVIDIA_H100
Kunlun_P800
TTFT P99 (ms)
ms
1534
5144
NVIDIA_H100
Kunlun_P800
TPOT P99 (ms)
ms
17.75
68.00
NVIDIA_H100
Kunlun_P800

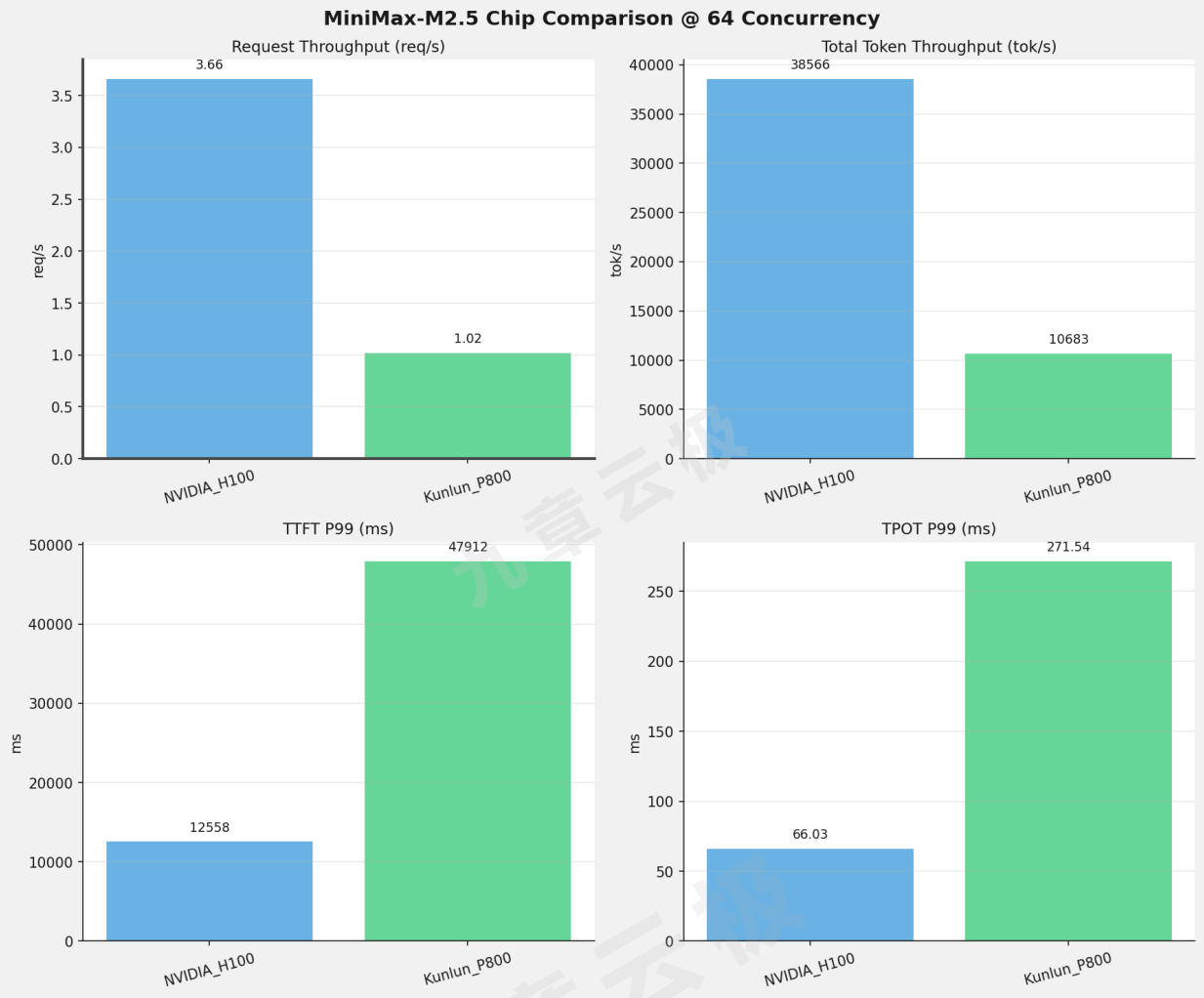
16并发


MiniMax-M2.5 Chip Comparison @ 16 Concurrency
Request Throughput (req/s)
req/s
2.51
0.74
Total Token Throughput (tok/s)
tok/s
26494
7731
TTFT P99 (ms)
ms
9917
2631
TPOT P99 (ms)
ms
92.41
23.92
NVIDIA_H100
Kunlun_P800

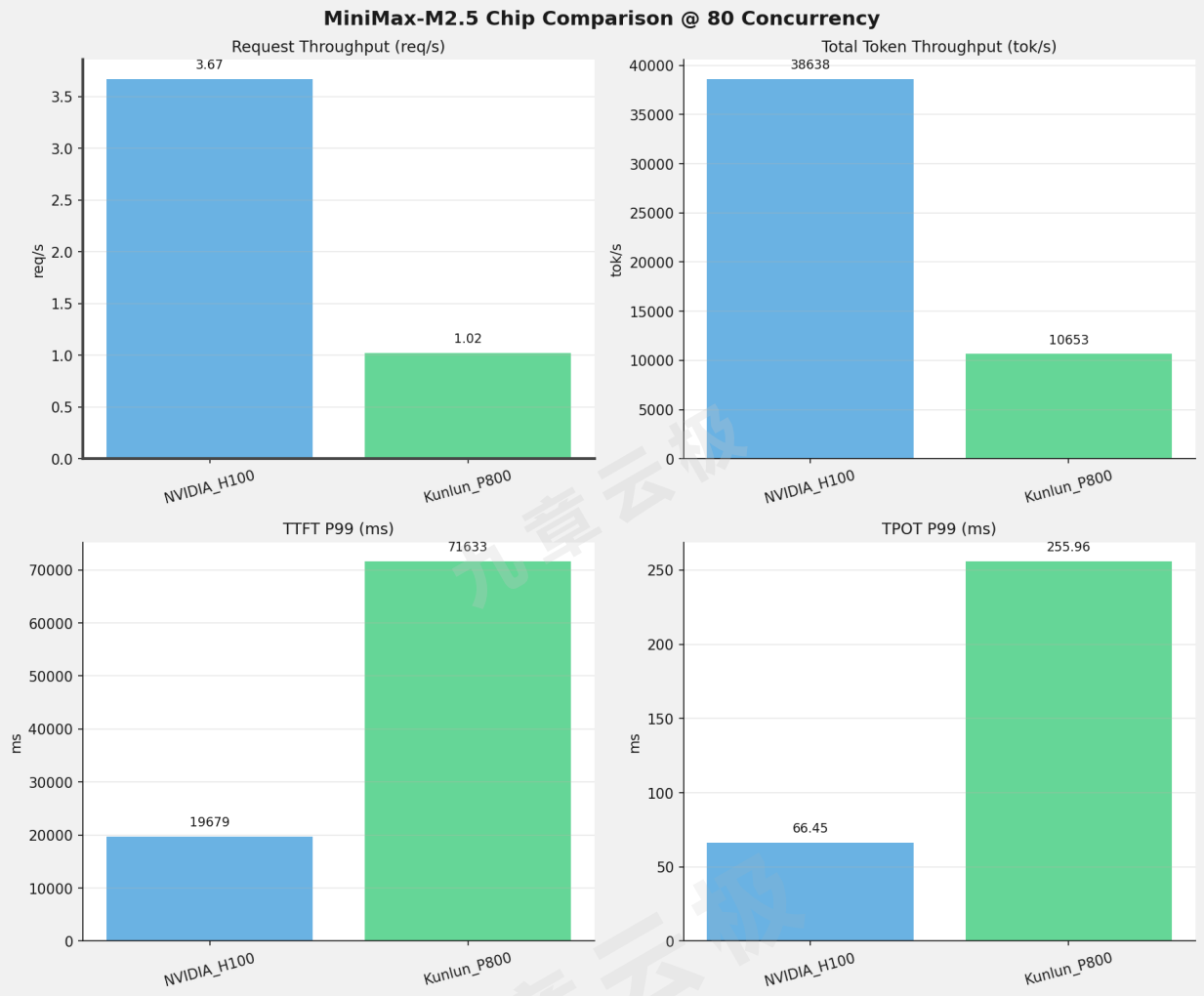
32并发


MiniMax-M2.5 Chip Comparison @ 32 Concurrency
Request Throughput (req/s)
req/s
3.12
0.91
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
32832
9513
TTFT P99 (ms)
ms
6557
22360
TPOT P99 (ms)
38.79
163.15

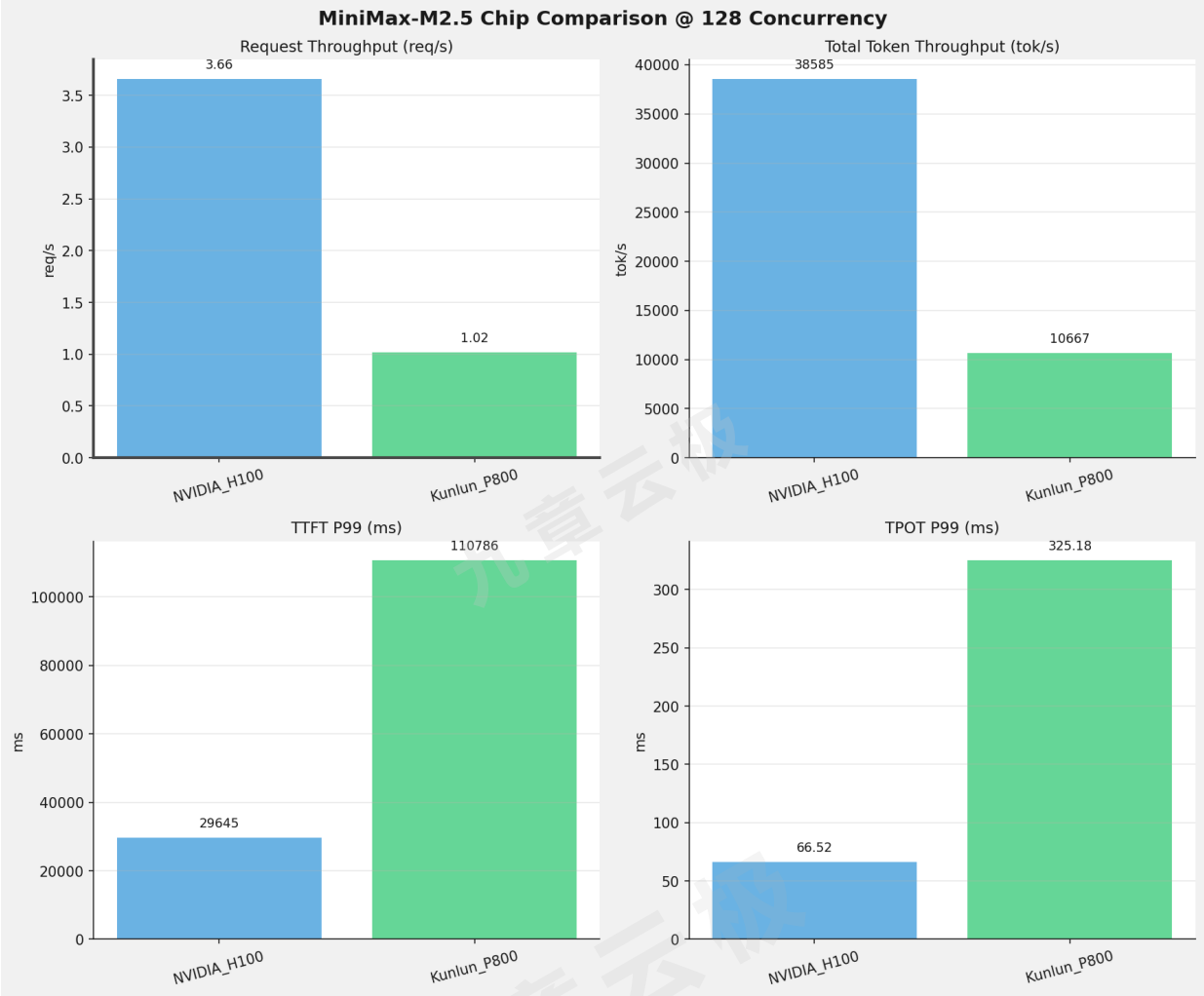
64并发


MiniMax-M2.5 Chip Comparison @ 64 Concurrency
Request Throughput (req/s)
req/s
3.66
1.02
Total Token Throughput (tok/s)
tok/s
38566
10683
TTFT P99 (ms)
ms
12558
47912
TPOT P99 (ms)
ms
66.03
271.54
NVIDIA_H100
Kunlun_P800

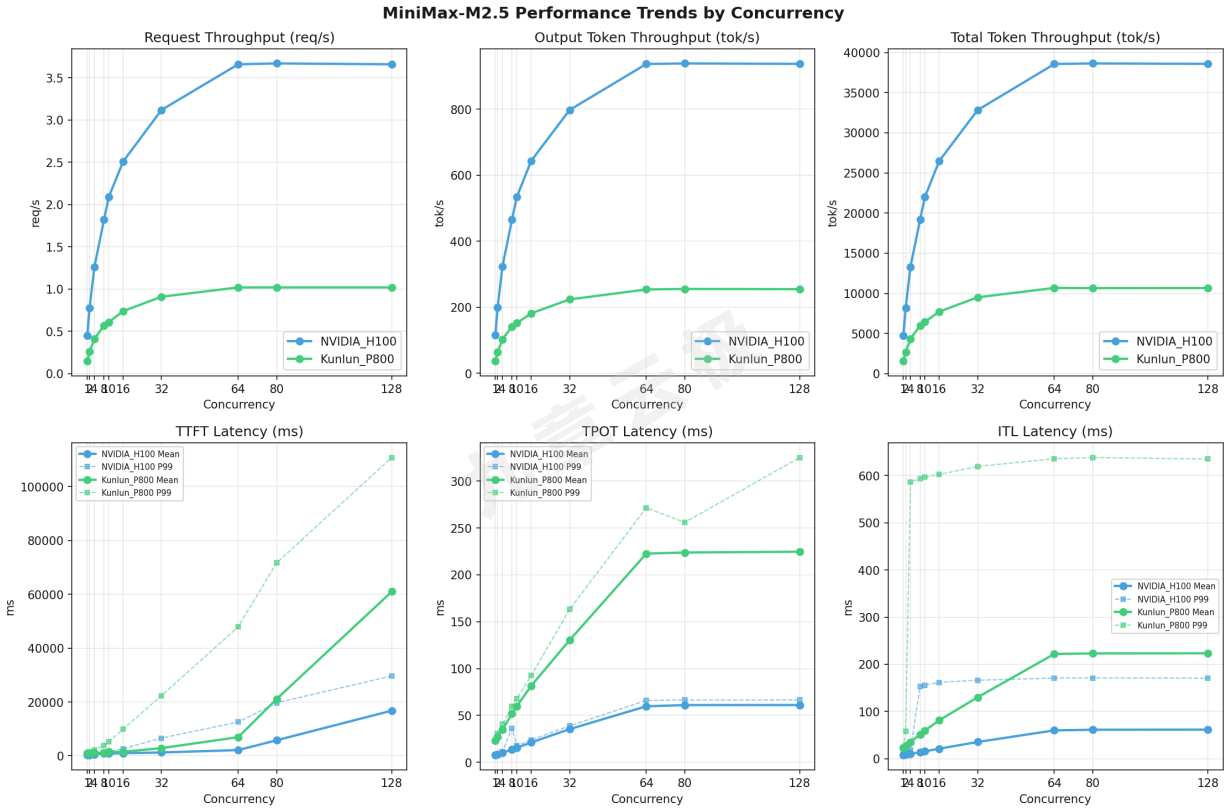
80并发


MiniMax-M2.5 Chip Comparison @ 80 Concurrency
Request Throughput (req/s)
req/s
3.67
1.02
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
38638
10653
TTFT P99 (ms)
ms
71633
19679
TPOT P99 (ms)
255.96
66.45

128并发



性能趋势对比图 (所有芯片)



各指标随并发级别性能对比详情

请求吞吐量（Request throughput (req/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	0.45	0.15	-0.30	-66.7%
2	0.78	0.26	-0.52	-66.7%
4	1.26	0.41	-0.85	-67.5%
8	1.82	0.57	-1.25	-68.7%
10	2.09	0.61	-1.48	-70.8%
16	2.51	0.74	-1.77	-70.5%
32	3.12	0.91	-2.21	-70.8%
64	3.66	1.02	-2.64	-72.1%
80	3.67	1.02	-2.65	-72.2%
128	3.66	1.02	-2.64	-72.1%

输出token吞吐量（Output token throughput (tok/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	115.31	37.20	-78.11	-67.7%
2	199.70	64.02	-135.68	-67.9%
4	323.45	102.96	-220.49	-68.2%
8	465.99	141.69	-324.30	-69.6%
10	534.52	152.16	-382.36	-71.5%
16	643.80	181.52	-462.28	-71.8%
32	797.81	223.77	-574.04	-72.0%
64	937.16	253.92	-683.24	-72.9%
80	938.91	255.42	-683.49	-72.8%
128	937.61	254.79	-682.82	-72.8%

总token吞吐量（Total token throughput (tok/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	4745.14	1585.87	-3159.27	-66.6%
2	8217.92	2705.12	-5512.80	-67.1%
4	13310.92	4342.80	-8968.12	-67.4%
8	19176.39	5980.27	-13196.12	-68.8%
10	21996.95	6448.31	-15548.64	-70.7%
16	26494.03	7730.83	-18763.20	-70.8%
32	32831.81	9513.02	-23318.79	-71.0%
64	38566.45	10682.97	-27883.48	-72.3%
80	38638.21	10653.29	-27984.92	-72.4%
128	38585.00	10666.52	-27918.48	-72.4%

首token延迟（P99 TTFT (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	286.01	921.76	+635.75	+222.3%
2	466.40	1591.52	+1125.12	+241.2%
4	826.89	2240.56	+1413.67	+171.0%
8	1364.44	3847.84	+2483.40	+182.0%
10	1534.23	5143.91	+3609.68	+235.3%
16	2630.84	9917.09	+7286.25	+277.0%
32	6556.86	22359.61	+15802.75	+241.0%
64	12557.76	47912.19	+35354.43	+281.5%
80	19679.20	71633.46	+51954.26	+264.0%
128	29645.32	110786.32	+81141.00	+273.7%

每token生成时间（P99 TPOT (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	7.68	23.38	+15.70	+204.4%
2	9.05	30.53	+21.48	+237.3%
4	11.41	40.56	+29.15	+255.5%
8	35.95	59.82	+23.87	+66.4%
10	17.75	68.00	+50.25	+283.1%
16	23.92	92.41	+68.49	+286.3%
32	38.79	163.15	+124.36	+320.6%
64	66.03	271.54	+205.51	+311.2%
80	66.45	255.96	+189.51	+285.2%
128	66.52	325.18	+258.66	+388.8%

token间延迟（P99 ITL (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	8.57	24.07	+15.50	+180.9%
2	16.54	58.36	+41.82	+252.8%
4	18.61	586.65	+568.04	+3052.3%
8	152.68	593.49	+440.81	+288.7%
10	155.84	596.54	+440.70	+282.8%
16	161.83	602.60	+440.77	+272.4%
32	166.38	619.41	+453.03	+272.3%
64	170.95	635.84	+464.89	+271.9%
80	171.22	638.32	+467.10	+272.8%
128	170.62	635.16	+464.54	+272.3%

测试场景二：超长上下文请求测试

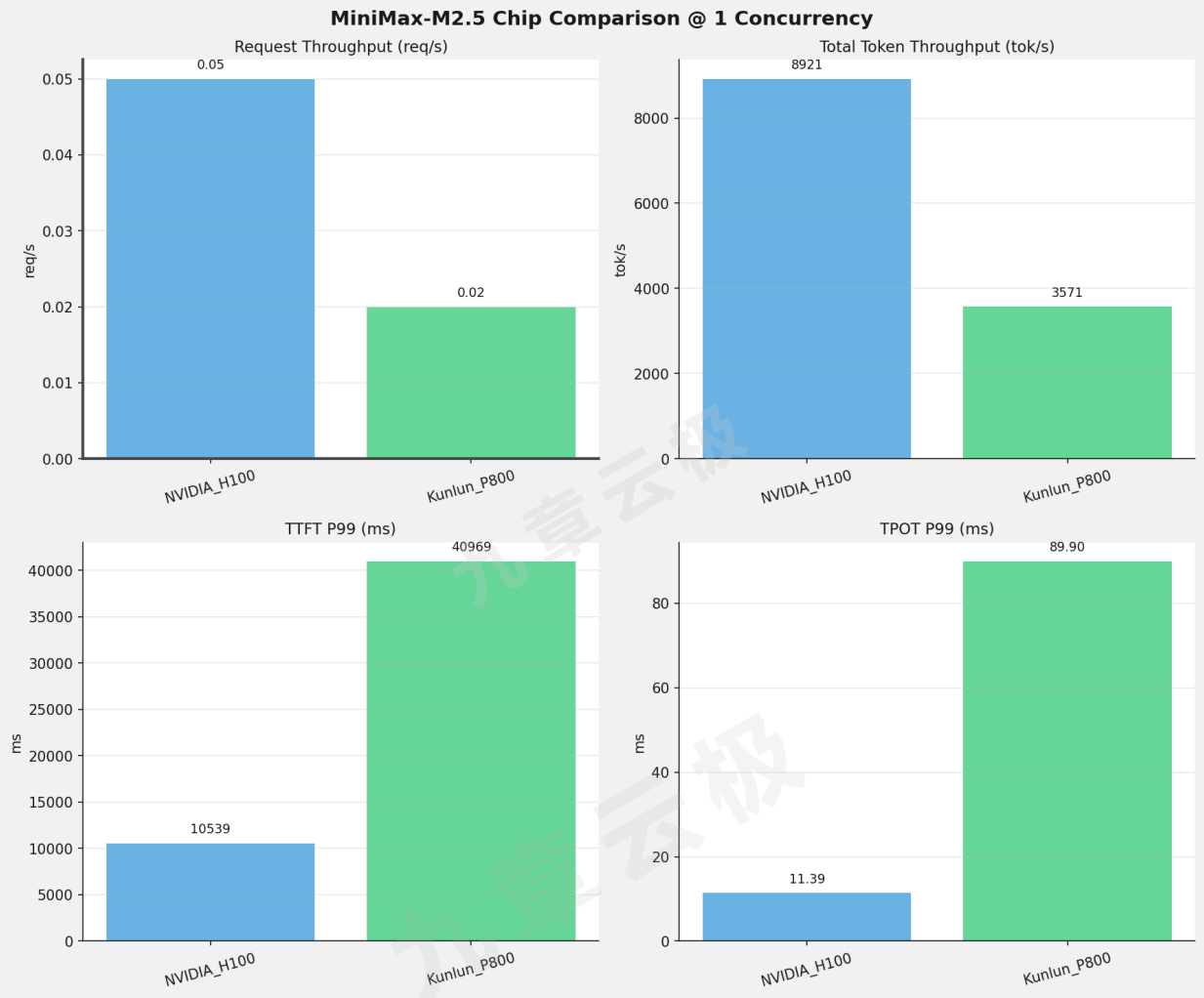
测试目标：对超长上下文的请求，使用vllm bench serve工具对并发数逐级增加场景的性能基准验证.

📊 测试概览

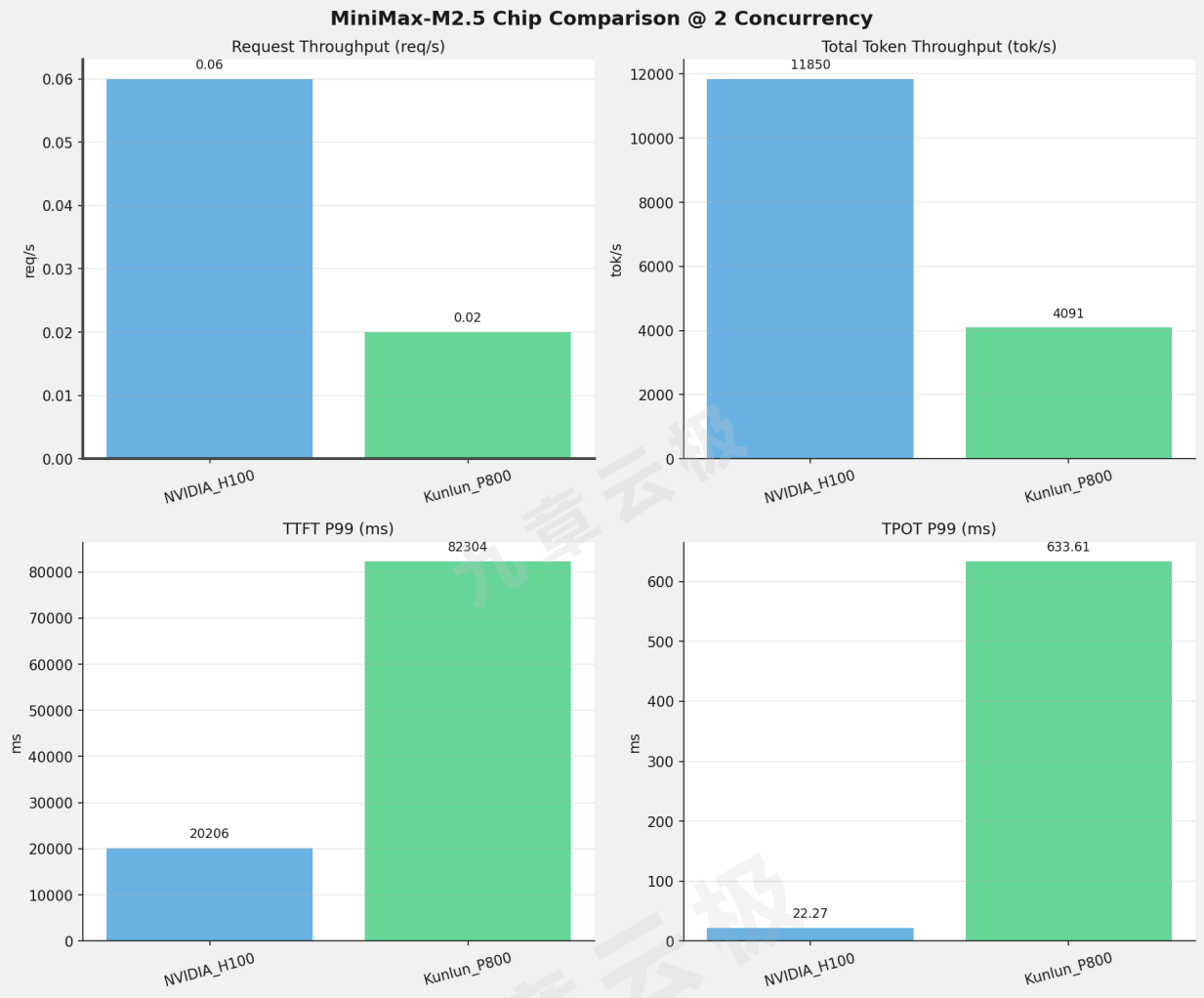
项目	配置	备注
数据集	random	
并发数	1, 2, 4, 8, 10	
总请求数	100	
请求输入上下文长度	194560（190k）	
请求输出上下文长度	1024（1k）	
被测芯片	NVIDIA_H100, Kunlun_P800	
被测模型	NVIDIA_H100 (MiniMax-M2.5); Kunlun_P800 (MiniMax-M2.5-W8A8-INT8-Dynamic)	

芯片性能对比柱状图

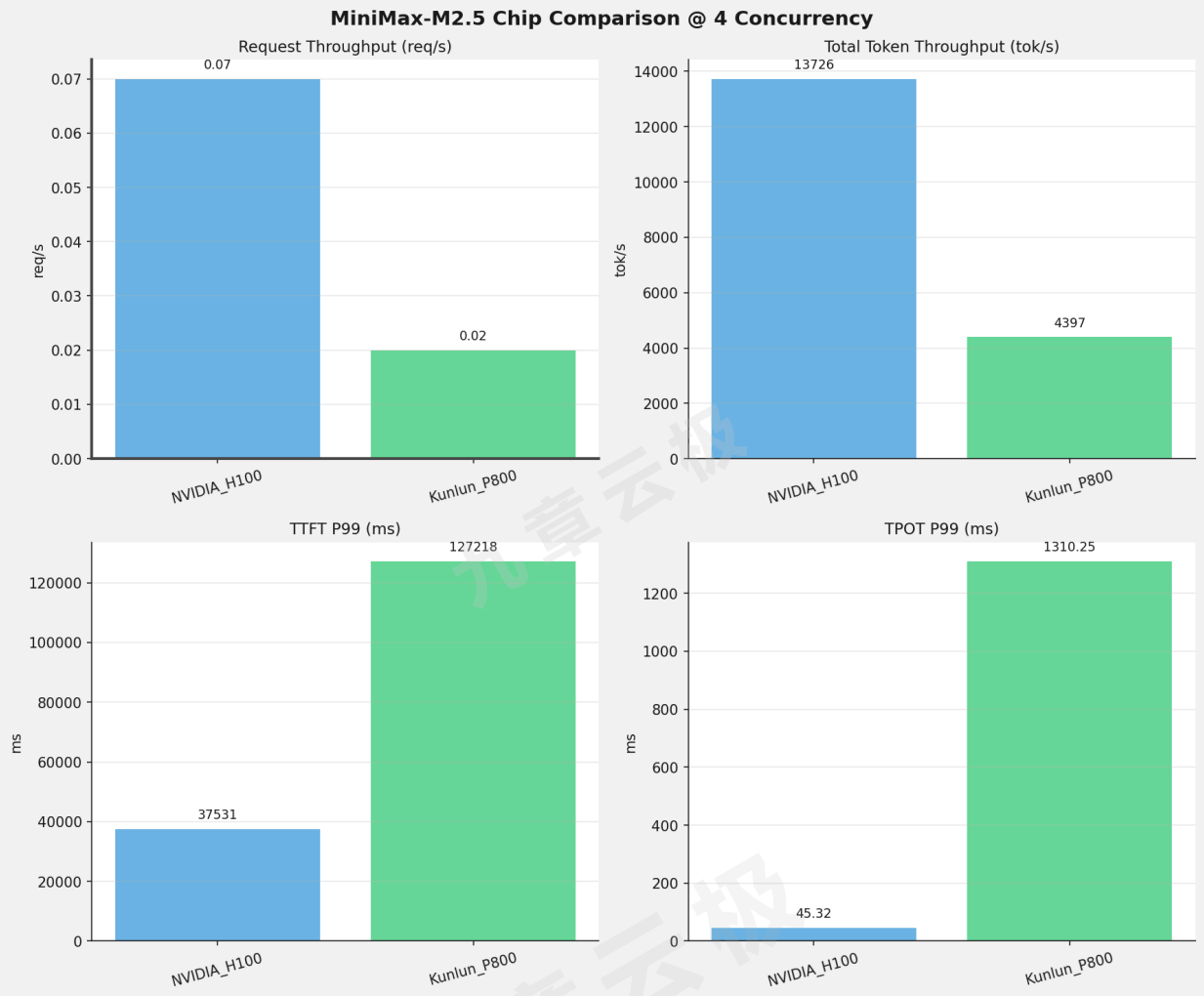
1并发



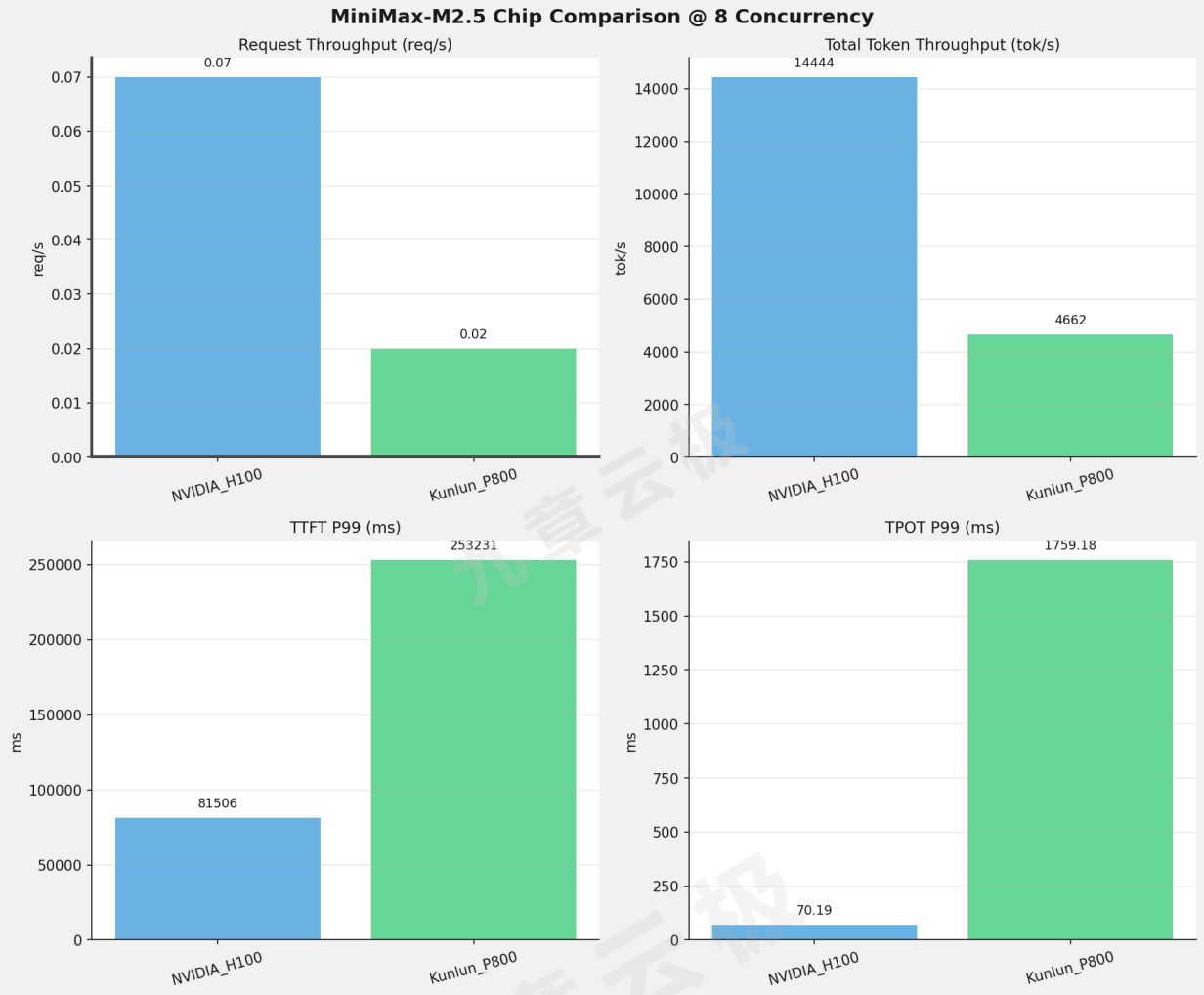
2并发


MiniMax-M2.5 Chip Comparison @ 2 Concurrency
Request Throughput (req/s)
req/s
0.06
0.02
Total Token Throughput (tok/s)
tok/s
11850
4091
TTFT P99 (ms)
ms
82304
20206
TPOT P99 (ms)
ms
633.61
22.27
NVIDIA_H100
Kunlun_P800

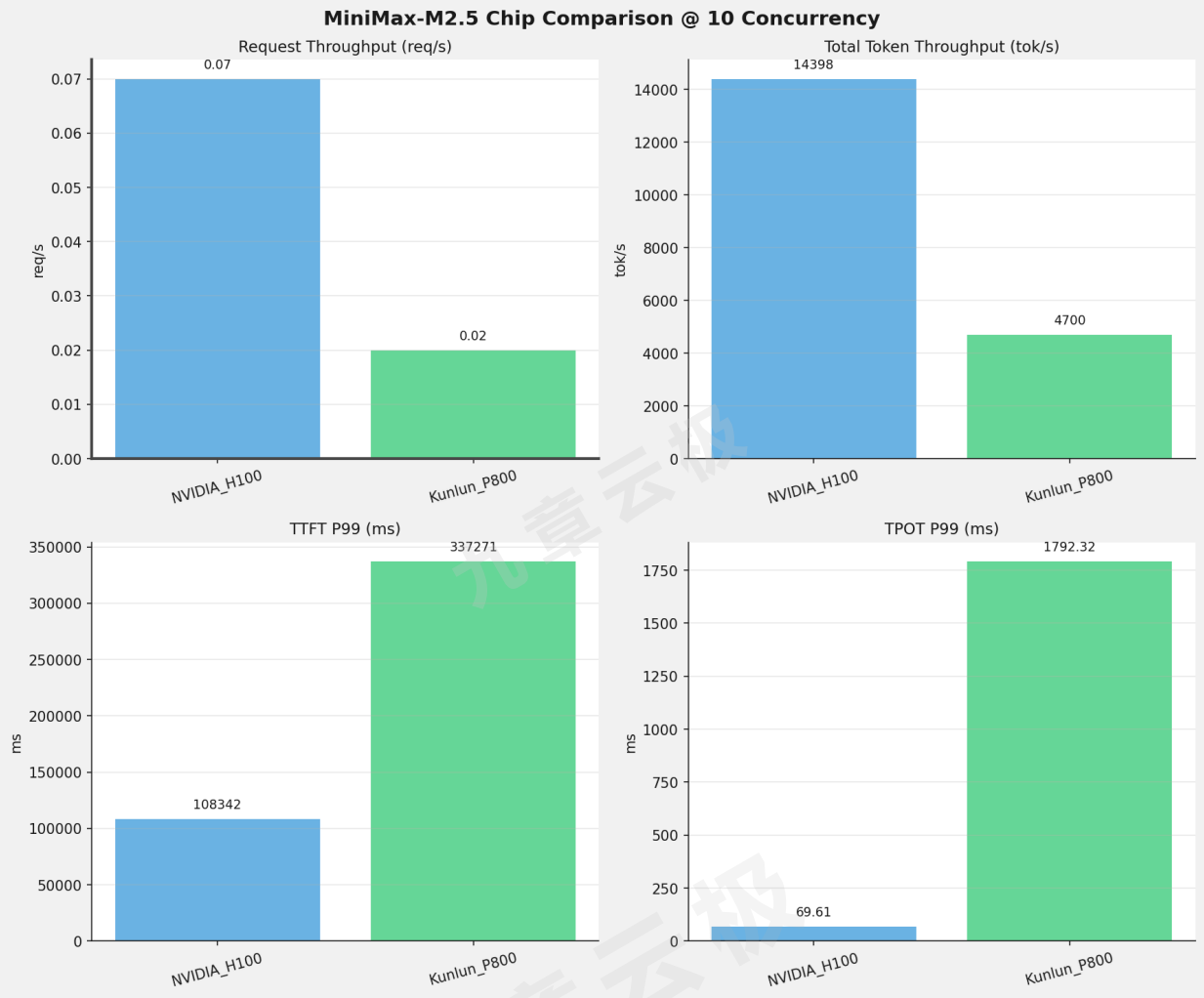
4并发


MiniMax-M2.5 Chip Comparison @ 4 Concurrency
Request Throughput (req/s)
req/s
0.07
0.02
Total Token Throughput (tok/s)
tok/s
13726
4397
TTFT P99 (ms)
ms
127218
37531
TPOT P99 (ms)
ms
1310.25
45.32
NVIDIA_H100
Kunlun_P800

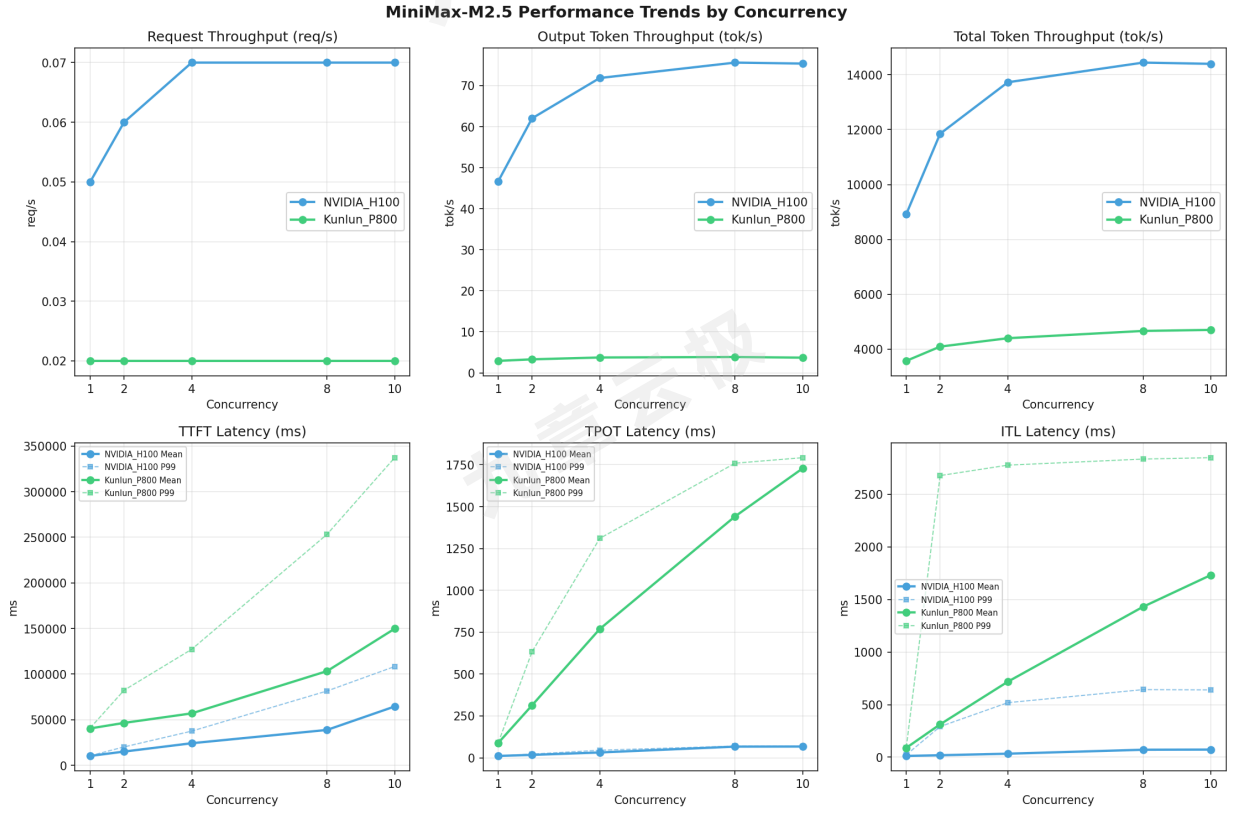
8并发


MiniMax-M2.5 Chip Comparison @ 8 Concurrency
Request Throughput (req/s)
req/s
0.07
0.02
NVIDIA_H100
Kunlun_P800
Total Token Throughput (tok/s)
tok/s
14444
4662
TTFT P99 (ms)
ms
253231
81506
TPOT P99 (ms)
1759.18
70.19

10并发



性能趋势对比图 (所有芯片)



📈 各指标随并发级别性能对比详情

请求吞吐量（Request throughput (req/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	0.05	0.02	-0.03	-60.0%
2	0.06	0.02	-0.04	-66.7%
4	0.07	0.02	-0.05	-71.4%
8	0.07	0.02	-0.05	-71.4%
10	0.07	0.02	-0.05	-71.4%

输出token吞吐量（Output token throughput (tok/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	46.70	2.92	-43.78	-93.7%
2	62.03	3.29	-58.74	-94.7%
4	71.85	3.74	-68.11	-94.8%
8	75.61	3.86	-71.75	-94.9%
10	75.37	3.70	-71.67	-95.1%

总token吞吐量（Total token throughput (tok/s)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	8921.40	3571.06	-5350.34	-60.0%
2	11850.06	4090.58	-7759.48	-65.5%
4	13726.45	4396.76	-9329.69	-68.0%
8	14443.68	4662.13	-9781.55	-67.7%
10	14398.41	4699.60	-9698.81	-67.4%

首token延迟（P99 TTFT (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	10539.10	40968.87	+30429.77	+288.7%
2	20205.87	82303.98	+62098.11	+307.3%
4	37530.99	127218.01	+89687.02	+239.0%
8	81506.35	253230.60	+171724.25	+210.7%
10	108342.29	337270.92	+228928.63	+211.3%

每token生成时间（P99 TPOT (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	11.39	89.90	+78.51	+689.3%
2	22.27	633.61	+611.34	+2745.1%
4	45.32	1310.25	+1264.93	+2791.1%
8	70.19	1759.18	+1688.99	+2406.3%
10	69.61	1792.32	+1722.71	+2474.8%

token间延迟（P99 ITL (ms)）

并发数	NVIDIA_H100	Kunlun_P800	差值	百分比
1	22.97	93.69	+70.72	+307.9%
2	291.30	2678.51	+2387.21	+819.5%
4	518.54	2777.75	+2259.21	+435.7%
8	642.87	2835.54	+2192.67	+341.1%
10	639.97	2848.90	+2208.93	+345.2%

测试场景三：长上下文高并发极限验证

测试目标：多并发长上下文的情况下，验证各芯片单节点同时能处理的最大请求数。

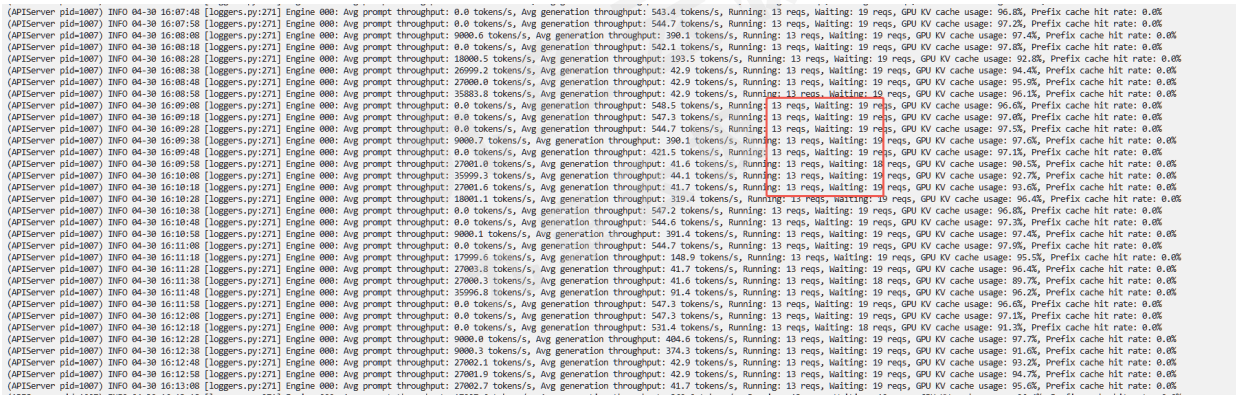
测试概览

项目	配置	备注
数据集	random	
并发数	32, 64	
总请求数	1000	
请求输入上下文长度	90000（约90k）	
请求输出上下文长度	2000（约2k）	
被测芯片	NVIDIA_H100, Kunlun_P800	
被测模型	NVIDIA_H100 (MiniMax-M2.5); Kunlun_P800 (MiniMax-M2.5-W8A8-INT8-Dynamic)	

监控处理请求极限

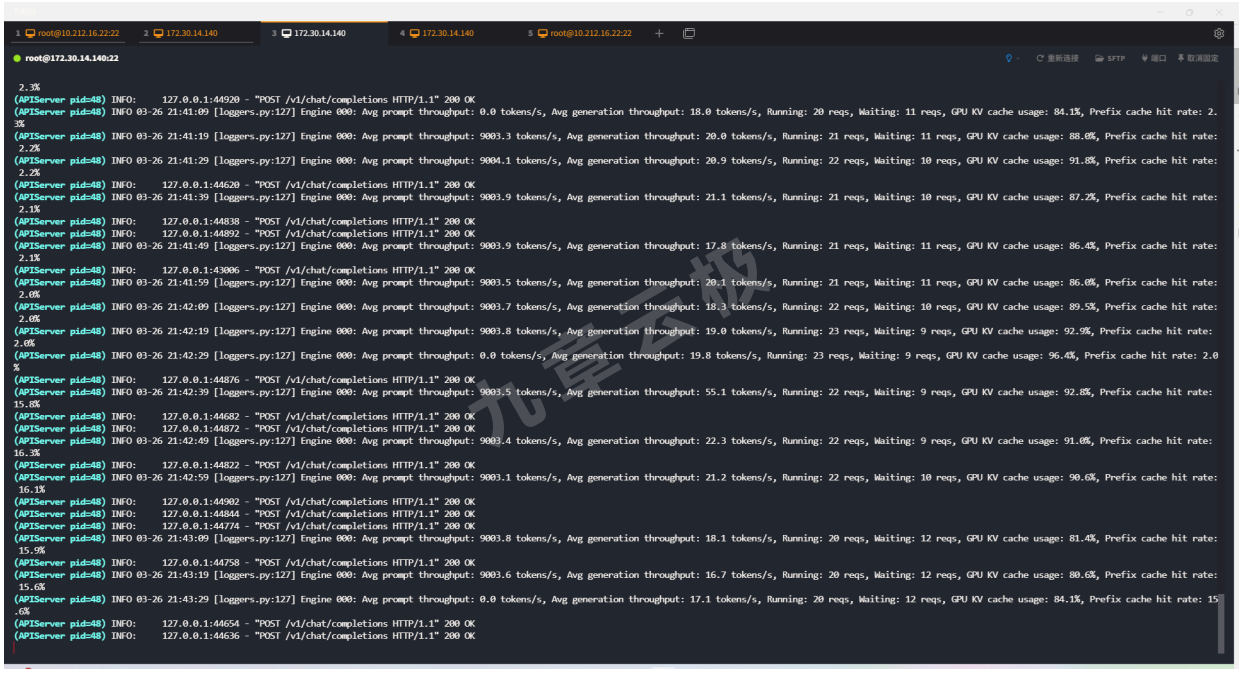
NVIDIA_H100芯片

测试平稳后，最大可同时处理请求数13个，且非常稳定，缓存命中率波动正常



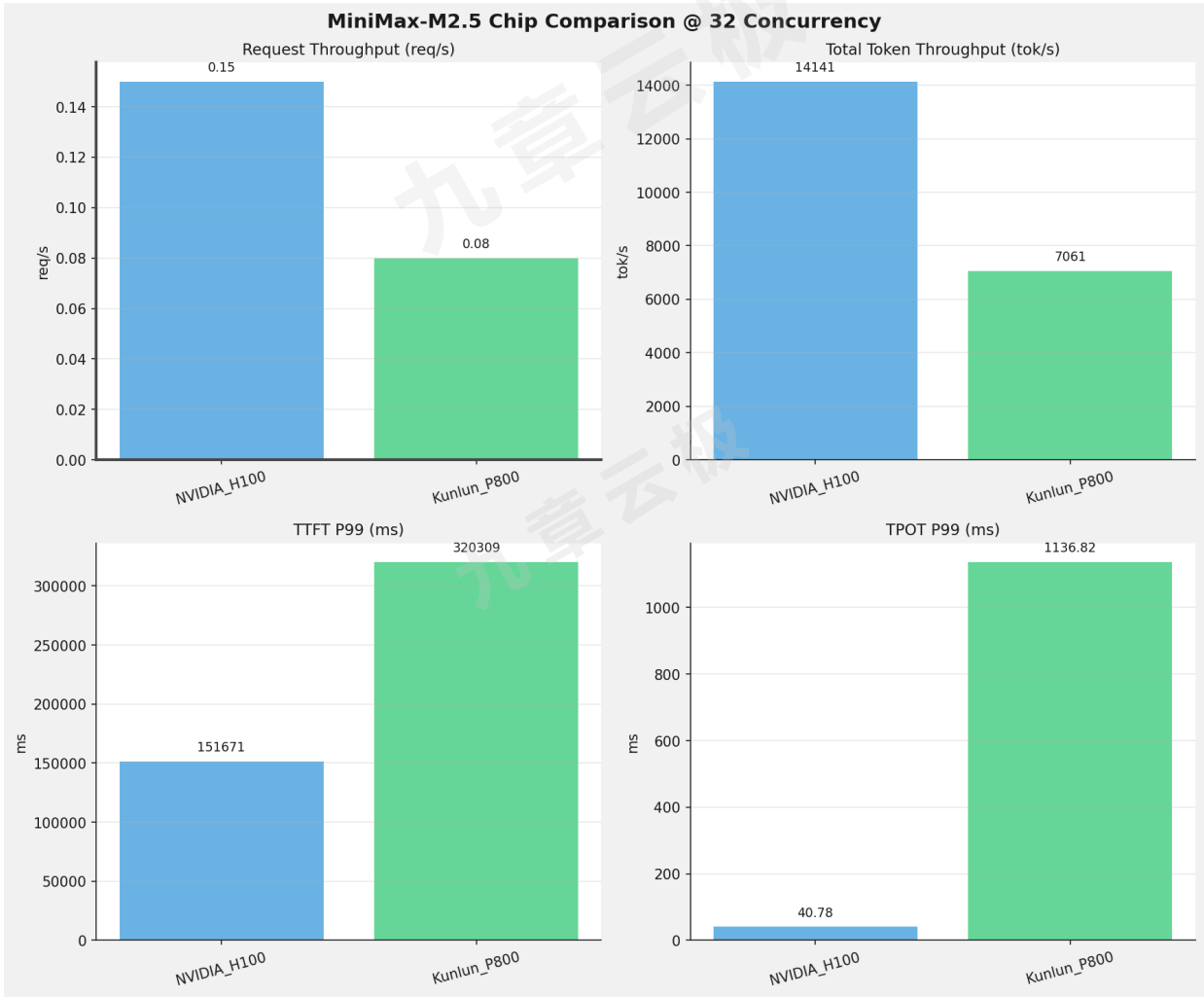
Kunlun_P800芯片

测试平稳后，最大可同时处理请求数在18~23之间波动，缓存命中率波动较大（在2%~16%之间）

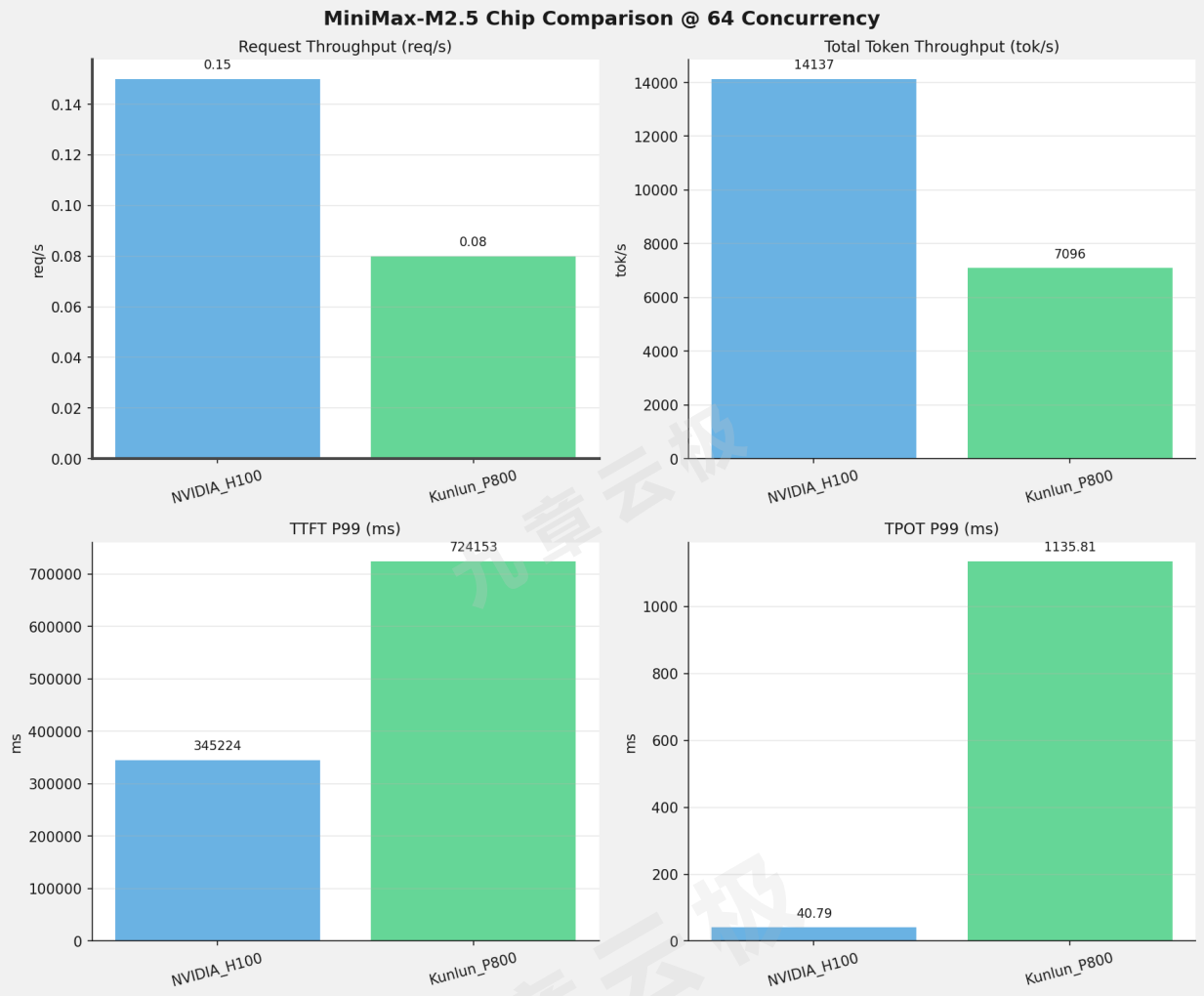


芯片性能对比柱状图

32并发



64并发



📈 各并发级别性能对比详情

32 并发

指标	NVIDIA_H100	Kunlun_P800
请求吞吐量（Request throughput (req/s)）	**0.15** ⭐	0.08
输出token吞吐量（Output token throughput (tok/s)）	**307.27** ⭐	20.03
总token吞吐量（Total token throughput (tok/s)）	**14140.63** ⭐	7060.97
首token延迟（P99 TTFT (ms)）	**151671.11** ⭐	320308.58
每token生成时间（P99 TPOT (ms)）	**40.78** ⭐	1136.82
token间延迟（P99 ITL (ms)）	**363.93** ⭐	1612.65

64 并发

指标	NVIDIA_H100	Kunlun_P800
请求吞吐量（Request throughput (req/s)）	**0.15** ⭐	0.08
输出token吞吐量（Output token throughput (tok/s)）	**307.19** ⭐	20.70
总token吞吐量（Total token throughput (tok/s)）	**14136.71** ⭐	7096.07
首token延迟（P99 TTFT (ms)）	**345223.92** ⭐	724153.37
每token生成时间（P99 TPOT (ms)）	**40.79** ⭐	1135.81
token间延迟（P99 ITL (ms)）	**364.12** ⭐	1612.99

测试场景四：多I/O测试

测试目标

测试不同输入输出长度和并发级别下的性能表现，分析同一芯片同一模型在不同输入输出长度和并发级别下的性能指标变化趋势。

📊 测试概览

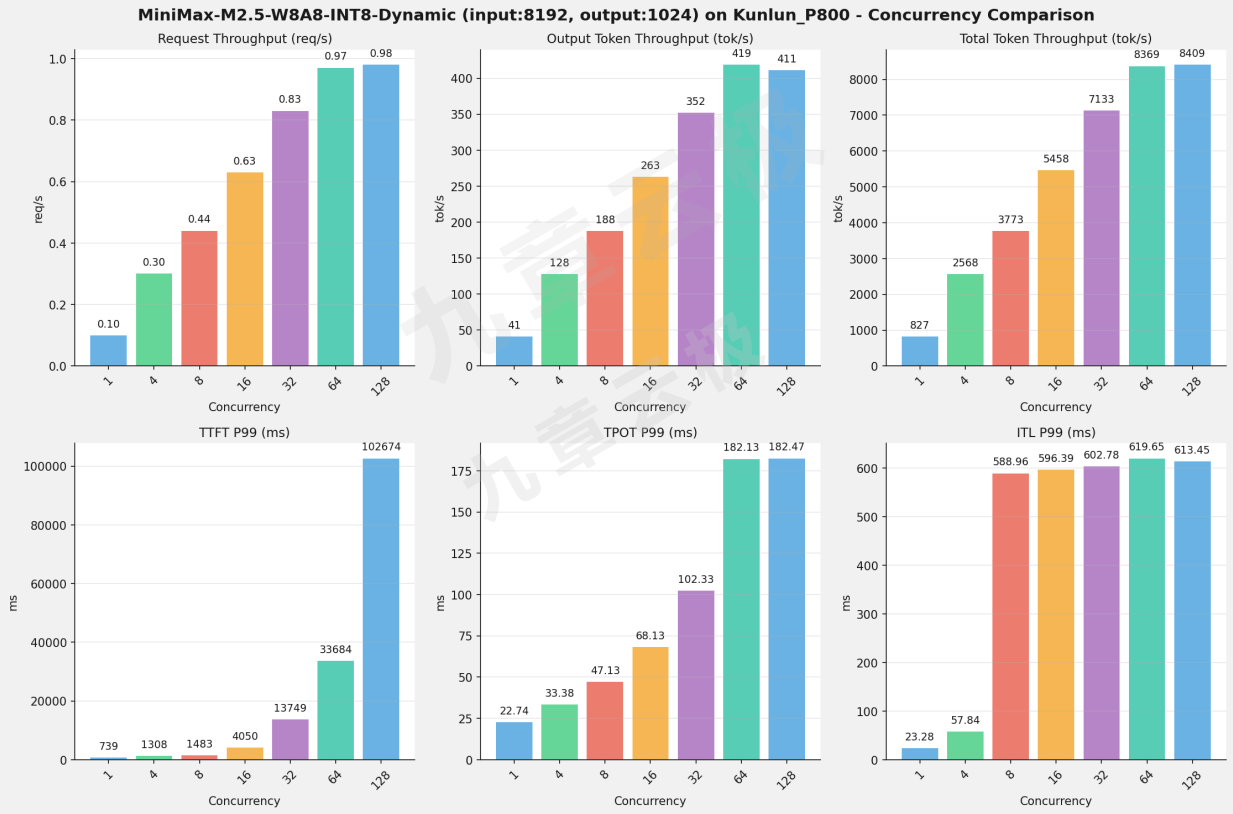
项目	配置	备注
数据集	random	
并发数	1, 4, 8, 16, 32, 64, 128	
总请求数	1000	
输入输出长度	(128, 128), (512, 256), (1024, 512), (2048, 1024), (4096, 2048), (8192, 1024)	
被测芯片	NVIDIA_H100, Kunlun_P800	
被测模型	NVIDIA_H100 (MiniMax-M2.5); Kunlun_P800 (MiniMax-M2.5-W8A8-INT8-Dynamic)	

📋 各I/O测试汇总（固定请求上下文长度，随并发数变化）

> **报告说明: 由于本测试场景比较多，此报告仅列出一组测试结果作为示例，且仅列出Kunlun_P800测试结果**

input: 8192, output: 1024

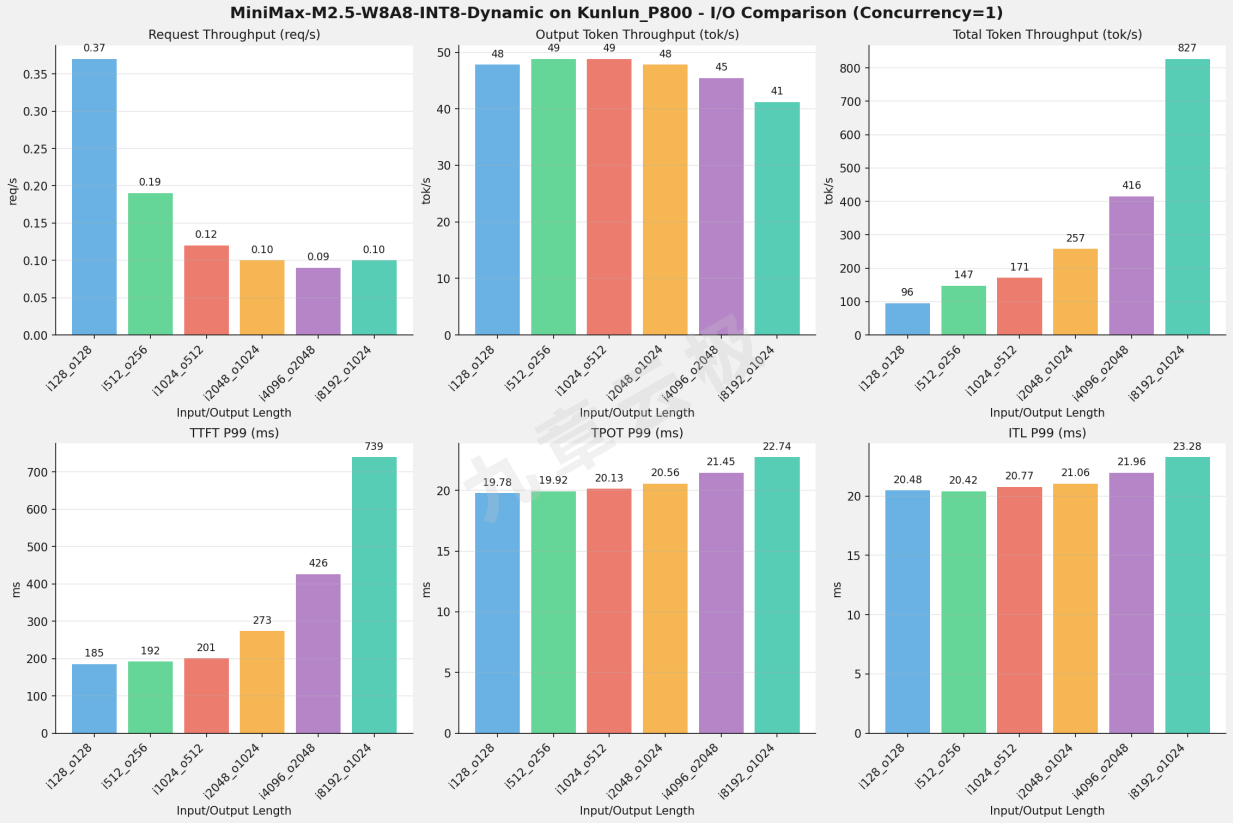
并发数	请求吞吐量 (req/s)	输出Token吞吐量 (tok/s)	总Token吞吐量 (tok/s)	TTFT P99 (ms)	TPOT P99 (ms)	ITL P99 (ms)
1	0.10	41.23	826.58	739.49	22.74	23.28
4	0.30	128.01	2567.54	1308.30	33.38	57.84
8	0.44	187.92	3772.55	1482.68	47.13	588.96
16	0.63	263.08	5457.67	4050.05	68.13	596.39
32	0.83	352.07	7133.48	13748.61	102.33	602.78
64	0.97	418.85	8369.48	33683.94	182.13	619.65
128	0.98	411.01	8408.86	102674.50	182.47	613.45



📊 I/O对比（固定并发数, 随请求上下文长度变化）

并发数 = 1

指标	i128_o128	i512_o256	i1024_o512	i2048_o1024	i4096_o2048	i8192_o1
请求吞吐量 (req/s)	0.37	0.19	0.12	0.10	0.09	0.10
输出 Token 吞吐量 (tok/s)	47.82	48.82	48.80	47.79	45.44	41.23
总 Token 吞吐量 (tok/s)	95.63	147.45	170.80	257.41	415.54	826.58
TTFT P99 (ms)	184.99	191.87	200.73	273.32	426.06	739.49
TPOT P99 (ms)	19.78	19.92	20.13	20.56	21.45	22.74
ITL P99 (ms)	20.48	20.42	20.77	21.06	21.96	23.28



测试场景五：模型精度测试

测试目标

模型精度测试目标主要是通过标准指标（如准确率、精确率、召回率、F1值、mAP、AUC）衡量模型在测试集上的输出与真实标签的一致性，评估其基本判别能力。

MiniMax-M2.5模型 - 各测试任务整体比对

Task	NVIDIA_H100(FP8)	Kunlun_P800(W8A8-INT8-Dynamic)	差值	百分比
IFBench (Strict)	0.6067	0.6233	0.0167	+ 2.75%
IFBench (Loose)	0.6433	0.6600	0.0167	+ 2.59%
lm-eval:gsm_plus (Flexible)	0.6863	0.7398	0.0535	+ 7.80%
lm-eval:gsm_plus (Strict)	0.7307	0.7251	-0.0056	- 0.77%
lm-eval:mmlu_pro	0.7378	0.6622	-0.0756	- 10.25%
lm-eval:ruler	0.5461	N/A	N/A	N/A

> - IFBench模型精度测试脚本参见《附录二》
> - lm-eval模型精度测试脚本参见《附录三》

测试场景六：多轮对话测试

📊 测试概览

测试工具：vllm 内置的多轮对话测试脚本

条目	值
并发客户端 (num_clients)	50
总对话轮数 (num_conversations)	1000
活跃对话数 (active_conversations)	100
输入轮数 (input_num_turns)	10
输入 Token 长度 (input_num_tokens)	200
输出 Token 长度 (output_num_tokens)	100
语料来源	推荐的语料 pg1184.txt
seed	0

和英伟达比对

指标	NVIDIA_H100	Kunlun_P800
总耗时	140.633 秒	459.850 秒
请求吞吐量（RPS）	28.087	8.168

Kunlun_P800多轮对话详细测试结果

指标	count	mean	std	99%	99.9%	max
ttft_ms	3756.0	1536.34	467.96	2206.48	2413.37	2414.27
tpot_ms	3756.0	46.22	3.32	54.47	58.02	59.86
latency_ms	3756.0	6102.88	383.59	6840.95	7009.35	7009.98
input_num_turns	3756.0	5.91	2.25	9.00	9.00	9.00
input_num_tokens	3756.0	932.00	335.38	1395.00	1395.00	1395.00
output_num_tokens	3756.0	99.89	0.46	100.00	100.00	100.00
output_num_chunks	3756.0	96.88	5.88	99.00	99.00	99.00

> 多轮对话测试脚本参见本报告《附录四》

测试场景七：基础推理能力验证

测试目标：

验证模型在芯片环境上的基础推理和兼容性的支持能力情况，可作为快速选型的一个基础指标。

测试说明

> 比对说明：本章节只列出在Kunlun_P800芯片平台上的测试结果

> 状态说明：✅ 已通过， ⏳ 未测试， ❌ 未通过， ⚠️ 部分通过

A. 基础推理能力

#	测试点	测试内容	状态	备注
A1	单轮对话	发送单条prompt，验证正常生成	✅	
A2	多轮对话	5轮对话，验证上下文保持和连贯性	✅	
A3	System Prompt	设置系统角色，验证模型遵循程度	✅	
A4	流式输出	stream=true，验证SSE逐token返回	✅	
A5	非流式输出	stream=false，验证完整返回	✅	
A6	Temperature 控制	temp=0 vs temp=1.0，验证输出差异	✅	
A7	Top-p/Top-k采样	不同top_p/top_k值，验证多样性控制	✅	
A8	Max Tokens限制	设置max_tokens，验证输出不超限	✅	
A9	Stop Sequences	设置stop token，验证截断	✅	
A10	Seed 可复现性	相同seed+temp=0，验证输出一致	✅	
A11	多语言能力	中/英/日/韩/法等多语言输入输出	✅	
A12	特殊Token处理	含emoji、代码块、数学符号、HTML标签	✅	

B. 高级生成功能

#	测试点	测试内容	状态	备注
B1	思考模式（Thinking）	开启thinking mode，验证返回思考链...	✅	
B2	非思考模式（Instant）	关闭thinking，验证无hidden th...	❌	关闭思考模式，依然有reasoning_content思考内容
B3	思考模式切换	同一会话内thinking↔non-think...	❌	使用默认temperature 0.7
B4	工具调用-单工具	定义单个function，验证模型正确调用并传参	✅	
B5	工具调用-多工具	定义多个function，验证模型选择正确的工具	❌	未正常调用多个工具
B6	工具调用-并行调用	单次回复中并行调用多个工具	✅	
B7	工具调用-多步链式	工具结果作为下一步输入，验证3+步链式执行	❌	未正常进行链式工具调用
B8	JSON Mode	response_format=json_ob...	✅	
B9	结构化输出	JSON Schema约束输出格式，验证字段完整性	✅	
B10	Prefix/Suffix约束	指定输出前缀或格式模板，验证遵循度	❌	

C. 多模态能力 （MiniMax-M2.5模型为文本模型，此测试组请跳过）

#	测试点	测试内容	状态
C1	单图理解	图片+文本提问	❌
C2	多图对比	跨图比较	❌
C3	高分辨率图片	4K分辨率	❌
C4	图表/OCR	表格截图	❌
C5	视频理解	视频文件	❌
C6	代码截图→代码	UI截图	❌
C7	多模态工具调用	图片触发工具	❌
C8	图片格式兼容性	PNG/JPEG/WebP	❌

D. 长上下文处理

#	测试点	测试内容	状态	备注
D1	短上下文基线	1K tokens	✅	
D2	中等上下文	8K-16K tokens	✅	
D3	长上下文	32K-64K tokens	✅	
D4	超长上下文	128K+ tokens	✅	
D5	大海捞针	NIAH	✅	
D6	上下文边界行为	max_model_len	✅	
D7	超出上下文截断	截断/拒绝	✅	
D8	长输出生成	4K-8K tokens	✅	

F. 稳定性与边界

#	测试点	测试内容	状态	备注
F1	空输入	空prompt	✅	返回Empty message handled:
F2	超大输入	超max_model_len	✅	返回Oversized input rejected: HTTPConnectionPool(host='127.0.0.1', port=8080): Read timed out. (read timeout=120)
F3	非法参数	temperature=-1, max_tokens=0,非法温度值：超过范围（>2）	✅	API request failed with status 400
F4	特殊字符注入	SQL/Prompt注入	✅	
F5	并发稳定性	200+并发（实际测试50并发）	✅	
F6	OOM恢复	显存耗尽	⏳	
F7	长时间运行	24小时	⏳	
F8	请求超时处理	超时断开	⏳	

G. API兼容性

#	测试点	测试内容	状态	备注
G1	OpenAI Chat Completions	/v1/chat/completions 接口兼容	✅	
G2	OpenAI Completions	/v1/completions 接口兼容	✅	
G3	模型列表	/v1/models 返回可用模型	✅	
G4	Usage 统计	usage 字段准确	✅	
G5	错误码规范	400/401/404/429/500 错误码	✅	
G6	客户端 SDK 兼容	Python openai / JS @ope...	⏳	
G7	响应格式变体	不同response_format	✅	
G8	Stream参数	stream参数测试	✅	

H. 质量评估

#	测试点	测试内容	状态	备注
H1	生成质量	质量对比	✅	
H2	生成一致性	多次生成	✅	
H3	幻觉率	事实错误	✅	
H4	指令遵循度	格式/角色	✅	
H5	响应相关性	问答相关性	✅	

I. 超长上下文验证

#	测试点	测试内容	状态	备注
I1	超长上下文（非流式）	验证超长上下文请求的非流式输出	✅	
I2	超长上下文（流式）	验证超长上下文请求的流式输出	✅	
I3	超长上下文（边界验证）	使用二分法逼近模型最大上下文长度	❌	未成功完成二分法验证
I4	超长上下文（思考模式）	验证超长上下文下 reasoning_conte...	✅	

附录一：benchmark执行脚本

执行命令

```
python run_benchmark.py --chip kunlun_p800 --model MiniMax-M2.5-W8A8-INT8-Dynamic --test-suite test_01,test_02,test_03,test_04,test_05

# 如果不指定test-suite参数，默认执行run_benchmark.py里TEST_SUITES定义的测试套件列表，可以根据自己需要修改默认测试套件
```

run_benchmark.py

```
import os
import yaml
import subprocess
import requests
import time
from datetime import datetime
from itertools import product
from pathlib import Path

API_KEY = os.environ.get("API_KEY", "abc123")

TEST_SUITES = ["test_01"]

RUN_ID = "01"

try:
    from gpu_monitor import GPUMonitor, generate_gpu_charts

    HAS_GPU_MONITOR = True
except ImportError:
    HAS_GPU_MONITOR = False
    print("Warning: GPU monitor module not available")


def get_model_info_from_api(base_url, api_key):
    headers = {"Authorization": f"Bearer {api_key}"}
    try:
        response = requests.get(f"{base_url}/v1/models", headers=headers, timeout=10)
        if response.status_code == 200:
            data = response.json()
            if data.get("data") and len(data["data"]) > 0:
                model_info = data["data"][0]
                model_name = model_info.get("id")
                owned_by = model_info.get("owned_by")
                model_path = model_info.get("root")
                if owned_by == "vllm" and model_path:
                    return model_name, model_path
                else:
```

```
                    return model_name, None
    except Exception as e:
        print(f"Failed to get model info from API: {e}")
    return None, None


def run_benchmark(chip_name, base_config, model_config, test_suites, run_id):
    base_url = base_config.get("base_url", "http://127.0.0.1:8080")

    model_name_yaml = model_config.get("name")
    served_model_name = model_config.get("served-model-name")
    model_path_yaml = model_config.get("model_path")

    model_name, model_path = get_model_info_from_api(base_url, API_KEY)

    if not model_name:
        model_name = served_model_name
    if not model_path:
        model_path = model_path_yaml

    print(f"Model Name: {model_name}")
    print(f"Model Path: {model_path}")
    print(f"Running test suites: {', '.join(test_suites)}")

    temperature = base_config.get("temperature", 0.7)
    seed = base_config.get("seed", 123)
    ready_timeout = base_config.get("ready-check-timeout-sec", 30)

    M = model_name_yaml
    output_base = f"reports/benchmark/{chip_name}/{M}"

    params_config = base_config.get("params", {})

    for test_suite in test_suites:
        test_params = params_config.get(test_suite, {})
        max_concurrency = test_params.get("max-concurrency", [10])
        num_prompts = test_params.get("num-prompts", [300])
        random_input_output_len = test_params.get(
            "random-input-output-len", [[20000, 100]]
        )

        run_id_dir = os.path.join(output_base, test_suite, run_id)
        if os.path.exists(run_id_dir):
            print(
                f"Error: Run ID '{run_id}' already exists for test suite
'{test_suite}' at path: {run_id_dir}"
            )
            print(f"Please either:")
            print(f"  1. Use a different RUN_ID (--run-id)")
            print(f"  2. Delete the existing directory: {run_id_dir}")
            continue

        print(f"\n=== Running test suite: {test_suite} ===")
```

```
    gpu_monitor = GPUMonitor(interval=10) if HAS_GPU_MONITOR else None

    for nc, np, io_len in product(
        max_concurrency, num_prompts, random_input_output_len
    ):
        ni = io_len[0]
        no = io_len[1]
        param_dir = f"{test_suite}/{run_id}/{nc}-{np}-i{ni}-o{no}"
        output_dir = os.path.join(output_base, param_dir)
        Path(output_dir).mkdir(parents=True, exist_ok=True)

        log_file = os.path.join(
            output_dir, f"bench-{test_suite}-{nc}-{np}-i{ni}-o{no}.log"
        )

        if gpu_monitor:
            monitor_param_dir = f"{test_suite}/{run_id}/{nc}-{np}-i{ni}-o{no}"
            gpu_monitor.start_monitoring(
                "monitor", chip_name, model_name_yaml, monitor_param_dir
            )

        cmd = [
            "vllm",
            "bench",
            "serve",
            "--backend",
            "openai-chat",
            "--endpoint",
            "/v1/chat/completions",
            "--dataset-name",
            test_params.get("dataset-name", "random"),
            "--random-input-len",
            str(ni),
            "--random-output-len",
            str(no),
            "--model",
            str(model_path),
            "--trust-remote-code",
            "--base-url",
            base_url,
            "--num-prompts",
            str(np),
            "--max-concurrency",
            str(nc),
            "--temperature",
            str(temperature),
            "--seed",
            str(seed),
            "--metric_percentiles",
            "95,99",
            "--served-model-name",
            str(model_name),
```

```
                "--ready-check-timeout-sec",
                str(ready_timeout),
            ]

            print(f"Running: {' '.join(cmd)}")
            print(f"Log file: {log_file}")

            log_f = open(log_file, "w")
            process = subprocess.Popen(
                cmd, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True
            )

            for line in process.stdout:
                print(line, end="")
                log_f.write(line)

            process.wait()
            log_f.close()

            if gpu_monitor:
                gpu_log = gpu_monitor.stop_monitoring()
                if gpu_log:
                    gpu_log_dir = os.path.dirname(gpu_log)
                    generate_gpu_charts(gpu_log, gpu_log_dir)

            print(f"Completed: {log_file}")
            time.sleep(30)


def main():
    import argparse

    parser = argparse.ArgumentParser(description="Run vLLM benchmark")
    parser.add_argument(
        "--chip",
        type=str,
        required=True,
        help="Chip name to test (e.g., hygon_bw1000, kunlun_p800, nvidia_h100)",
    )
    parser.add_argument(
        "--model",
        type=str,
        default=None,
        help="Model name to test (e.g., MiniMax-M2.5, Qwen3.5-122B-A10B). If not
specified, uses the first model in config.",
    )
    parser.add_argument(
        "--test-suite",
        type=str,
        default=None,
        help=f"Test suite to run (default: all). Available: {',
'.join(TEST_SUITES)}",
    )
```

```
    parser.add_argument(
        "--run-id",
        type=str,
        default=RUN_ID,
        help=f"Run ID to identify this test run (default: {RUN_ID})",
    )
    args = parser.parse_args()

    yaml_path = os.path.join(
        os.path.dirname(__file__), "config", "models_scenarios.yaml"
    )

    with open(yaml_path, "r", encoding="utf-8") as f:
        config = yaml.safe_load(f)

    base_config = config.get("base_config", {})
    params_config = base_config.get("params", {})
    models = config.get("models", {})

    chip_name = args.chip.lower()
    if chip_name not in models:
        print(
            f"Error: Chip '{chip_name}' not found in config. Available chips: {',
'.join(models.keys())}"
        )
        return

    available_models = models[chip_name]

    if args.model:
        model_name_lower = args.model.lower()
        selected_model = None
        for m in available_models:
            if m.get("name", "").lower() == model_name_lower:
                selected_model = m
                break
        if not selected_model:
            print(
                f"Error: Model '{args.model}' not found for chip '{chip_name}'.
Available models:"
            )
            for m in available_models:
                print(f"  - {m.get('name')} (served: {m.get('served-model-name')})")
            return
        model_configs = [selected_model]
    else:
        model_configs = [available_models[0]]
        print(f"No model specified, using default: {model_configs[0].get('name')}")

    test_suites_to_run = []
    if args.test_suite:
        test_suites_to_run = [s.strip() for s in args.test_suite.split(",")]
    else:
```

```
        test_suites_to_run = TEST_SUITES

    invalid_suites = [s for s in test_suites_to_run if s not in params_config]
    if invalid_suites:
        print(
            f"Error: Test suite(s) {invalid_suites} not found in config. Available:
{', '.join(params_config.keys())}"
        )
        return

    run_id = args.run_id

    for model_config in model_configs:
        print(f"Processing chip: {chip_name}, model: {model_config.get('name')}")
        run_benchmark(chip_name, base_config, model_config, test_suites_to_run,
run_id)
        print(f"Finished chip: {chip_name}, model: {model_config.get('name')}")


if __name__ == "__main__":
    main()
```

附录二：IFBench精度测试脚本

以MiniMax-M2.5-W8A8模型为例

ifbench_mm25_w8a8.sh

```
#!/bin/bash
ROOT_PATH=$(cd `dirname $0`; pwd)

echo $ROOT_PATH
cd ${ROOT_PATH}

CurDate=`date +'%Y%m%d'`
export NLTK_DATA=${ROOT_PATH}/nltk_data

cat > .env << 'EOF'
api_base=http://127.0.0.1:8000/v1
api_key=abc123
model=minimax-m2.5
temperature=1.0
top_p=0.95
top_k=40
max_tokens=8192
seed=42
input_file=data/IFBench_test.jsonl
output_file=data/mm25-responses.jsonl
workers=32
```

```
EOF

# 2. 生成模型响应
uv run python generate_responses.py

# 3. Thinking 模型后处理（重要！）
uv run python postprocess_thinking.py data/mm25-responses.jsonl -o data/mm25-
clean.jsonl

# 4. 运行评估
uv run python -m run_eval \
    --input_data=data/IFBench_test.jsonl \
    --input_response_data=data/mm25-clean.jsonl \
    --output_dir=eval
```

附录三： lm-eval精度测试脚本

以MiniMax-M2.5-W8A8模型为例

lm_eval_test.sh

```
#!/bin/bash
ROOT_PATH=$(cd `dirname $0`; pwd)

echo $ROOT_PATH
cd ${ROOT_PATH}

CurDate=`date +'%Y%m%d'`

export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1

#export HF_ENDPOINT=https://hf-mirror.com

ADDR=${ADDR:-127.0.0.1}
PORT=${PORT:-8000}
API_KEY=${API_KEY:-abc123}
LLM_ADDR="http://$ADDR:$PORT"

# 自动获取模型名和 tokenizer 路径
#MODEL_NAME=$(curl -s --header "Authorization: Bearer $API_KEY" $LLM_ADDR/v1/models
| jq -r .data[0].id)
#MODEL_PATH=$(curl -s --header "Authorization: Bearer $API_KEY" $LLM_ADDR/v1/models
| jq -r .data[0].root)
MODEL_NAME="minimax-m2.5"
LOCAL_MODEL_PATH="/data/MiniMax-M2.5-W8A8-INT8-Dynamic"

# model_args 构造
```

```
MODEL_ARGS_BASE_1="
{\"model\":\"$MODEL_NAME\",\"base_url\":\"$LLM_ADDR/v1/completions\",\"max_length\":
131072,\"tokenizer\":\"$LOCAL_MODEL_PATH\",\"trust_remote_code\":true,\"num_concurre
nt\":10,\"max_retries\":3,\"timeout\":12000,\"tokenized_requests\":false,\"headers\"
:{\"Authorization\":\"Bearer $API_KEY\"}}"
MODEL_ARGS_BASE_2="
{\"model\":\"$MODEL_NAME\",\"base_url\":\"$LLM_ADDR/v1/completions\",\"max_length\":
192512,\"tokenizer\":\"$LOCAL_MODEL_PATH\",\"trust_remote_code\":true,\"num_concurre
nt\":10,\"max_retries\":3,\"timeout\":12000,\"tokenized_requests\":false,\"headers\"
:{\"Authorization\":\"Bearer $API_KEY\"}}"

# 运行单个任务的函数
run_task_1() {
    local task_name=$1
    local max_tokens=$2
    local temperature=$3
    local unsafe_code=$4

    local do_sample="false"
    [ "$temperature" = "1.0" ] && do_sample="true"

    GEN_KWARGS="
{\"max_gen_toks\":$max_tokens,\"do_sample\":$do_sample,\"temperature\":$temperature,
\"top_p\":0.95,\"top_k\":40}"

    local unsafe_flag=""
    [ "$unsafe_code" = "true" ] && unsafe_flag="--confirm_run_unsafe_code" && export
HF_ALLOW_CODE_EVAL=1

    lm_eval \
        --model local-completions \
        --tasks $task_name \
        --output_path ./output/${task_name}/${MODEL_NAME}_${CurDate} \
        --model_args "$MODEL_ARGS_BASE_1" \
        --batch_size auto \
        --gen_kwargs "$GEN_KWARGS" \
        $unsafe_flag
}


run_task_2() {
    local task_name=$1
    local max_tokens=$2
    local temperature=$3
    local unsafe_code=$4

    local do_sample="false"
    [ "$temperature" = "1.0" ] && do_sample="true"

    GEN_KWARGS="
{\"max_gen_toks\":$max_tokens,\"do_sample\":$do_sample,\"temperature\":$temperature,
\"top_p\":0.95,\"top_k\":40}"
```

```
    local unsafe_flag=""
    [ "$unsafe_code" = "true" ] && unsafe_flag="--confirm_run_unsafe_code" && export
HF_ALLOW_CODE_EVAL=1

    lm_eval \
        --model local-completions \
        --tasks $task_name \
        --output_path ./output/${task_name}/${MODEL_NAME}_${CurDate} \
        --model_args "$MODEL_ARGS_BASE_2" \
        --batch_size auto \
        --limit 32 \
        --gen_kwargs "$GEN_KWARGS" \
        $unsafe_flag
}


run_task_1 mmlu_pro 8192 0.0 false

sleep 120
run_task_1 gsm_plus 8192 0.0 false

sleep 120
run_task_2 ruler 8192 0.0 false
```

附录四： 多轮对话测试命令

```
python benchmark_serving_multi_turn.py \
--model /data/MiniMax-M2.5-W8A8-INT8-Dynamic \
--served-model-name minimax-m2.5 \
--url http://127.0.0.1:8080/v1 \
--input-file generate_conversations.json \
--output-file generate_conversations_output.json \
--num-clients 50 \
--max-active-conversations 100 \
--warmup-step \
--extra-body-json {"chat_template_kwargs": {"enable_thinking": false}}
```